



Orbit One

Your life, nothing slips

Ground plan for the proof of concept

This document does two jobs. It explains the product vision well enough to pitch, and it gives a build plan for an Android-first POC that doubles as a portfolio piece for senior-engineer roles.

Owner: Pavan · Draft v2 · 26 September 2026

Clickable prototype: techhub-startup.github.io/orbit-one-prototype

Contents

1. Vision and positioning
2. Scope and phasing
3. Onboarding: demo first, permissions later
4. POC plan and timeline
5. Honest build risks
6. Architecture
7. Stack and why
8. AI agent design (explained from scratch)
9. Security and compliance baseline (day one)
10. Security architecture — read path
11. Consent versioning
12. Trust and transparency
13. Free-to-paid upgrade path per service
14. Monetization (next steps, not POC scope)
15. Portfolio angle and resume line
16. On-device analytics: is it working for real users?
17. Performance requirements (first-class)
18. Future feed improvements (post-POC, not in week 1 scope)
19. Home screen structure: two layers (proposed for the POC)
20. Demo flow and future feature introductions
21. Motion and transition principles (design principle for the POC and beyond)
Appendix: open decisions

1. Vision and positioning

One-line summary. An Android-first, consent-first personal assistant that keeps track of everything your phone already knows about your life. It reads the messages, emails, and app notifications you allow (bills, travel bookings, deliveries, appointments, renewals, sales), turns them into one clean list of what's coming up, reminds you at the right time, and notices when each thing is done. It does this processing on your phone.

App name: Orbit One (official from 26 Sep 2026; it replaces the working title "Your life, nothing slips").

Tagline: *Your life, nothing slips.*

The problem. The important moments of daily life arrive as scattered signals: an SMS from the electricity board, a bus booking email from Redbus, a courier's "out for delivery" notification, a clinic's appointment reminder, an insurance renewal notice, and five near-identical teasers for the same festival sale. People miss things because the information is spread across apps and buried in noise, not because it's missing.

What the app does

- Turns scattered signals into one list: **Today**, **Upcoming**, and **Overdue**, with each item explained ("why is this here?").
- Recognises that five messages about the same thing are one item. That covers the SMS, email, and app notification for one bill, and a week of reworded teasers for one Amazon sale.
- Reminds you before it matters, and follows up only if the thing isn't done yet (the bill isn't paid, the bus hasn't departed).
- Learns what you care about from simple feedback ("not a duplicate", "ignore this app").

Why finance is the first vertical. Money items have the clearest value: a missed bill or premium has a real cost. They also have the most regular formats (Indian bank, biller, SIP, and insurance messages), and "done" can be detected objectively through a "paid / debited" confirmation. That makes finance the right place to prove the data pipeline and measure accuracy before widening to the rest of life.

How the POC starts: app notifications only. The POC's only live data source is app notifications from apps the user picks. They arrive in real time, the user can see exactly which apps are included, and no email or SMS inbox is collected. Bank and UPI SMS still show up as notifications from the phone's SMS app, so much of the finance vertical is covered from day one. Reading the SMS inbox and Gmail come later, as separate opt-in phases. The demo mode shows the life-wide vision with sample data.

What makes it different

- **Demo first, permissions later.** Users see the full experience with clearly labelled sample data before being asked for any access (section 3).
- **On-device first.** The on-device agent classifies and parses notification content on the phone and then discards it; nothing raw leaves the phone. Cloud AI is off by default, and when turned on it receives masked snippets only (section 8).
- **Consent you can audit.** Every grant is one source at a time, recorded in a versioned ledger, and revocable in one tap (sections 11 and 12).
- **Privacy is never a paid upgrade, and user data is never sold** (section 14).

2. Scope and phasing

Phase	What	Real data?	When
Vision	One assistant for the user's whole digital life: finance, travel, deliveries, appointments, renewals, sales and events, and any app the user allowlists	Yes, vertical by vertical	Long term
POC (P0)	Demo mode across all verticals (sample data). One live source: app notifications from a user-chosen allowlist, processed on device. Real extraction for finance only (bills, SIPs, insurance premiums, payment confirmations). Consent ledger, reminders, dedup, audit, export and delete, cloud AI fallback, sync, Free/Pro gating, vault basics	Finance only, from notifications	4 weeks full-time, committed base (section 4)
P0 stretch	Sales and events as an opt-in category (the Amazon teaser case) on real data; a second vertical plug-in (travel bookings) to prove the plug-in design	Opt-in	If ahead of plan
P1: more sources (Phase 2)	Gmail (read-only) first, then SMS inbox reading (history and full text of bank SMS), each a separate opt-in with its own consent scope and the platform reviews in section 5.1. Not needed to ship the POC; it improves how many items are caught	Yes, per opt-in	Last; optional weeks 5–6, public offer after the trigger (section 4)
P1: second verticals	Travel bookings and deliveries, then appointments and renewals, each as a plug-in with its own consent scope. Basic payment history and persona	Yes, per opt-in	After the POC
P1: documents	Encrypted document vault (insurance, warranty, tickets) with extraction of due dates and clauses as suggestions the user accepts	Yes, per document	After the POC
P2	Bill payment through a licensed provider; regulated finance data routes (section 5.3); multi-device sync, web dashboard, family accounts; iOS client (Kotlin Multiplatform for shared domain logic)	Yes	Later

POC scope in detail

Feature	In POC?
Demo mode: full journey with labelled sample data (finance, travel, delivery, appointment, sale), no permissions needed	Yes (week 1)
Sign-in (Firebase Auth or Auth0, decided in week 1)	Yes
Onboarding that asks for one live source: notification access, with a per-app allowlist	Yes
Gmail and SMS inbox reading	No (Phase 2: Gmail first, then SMS inbox; built last and not blocking)
Finance extraction: bills, SIP debits, insurance premiums, due dates, amounts, payee	Yes
Life feed: Today / Upcoming / Overdue, plus manual items	Yes
Dedup across notifications from different apps (for example the SMS app's bank alert and the biller's own app for one bill)	Yes
Payment tracking: mark paid manually, or auto-match a later "payment successful / debited" notification	Yes
Manual quick-add for bills already known (because notifications give no history)	Yes
Local reminders and follow-up nudges (WorkManager)	Yes
Audit log, privacy dashboard (basic), export and delete-my-data	Yes
Labelled test set of masked real notifications and an accuracy report	Yes (this is the main de-risk item)
Sales/events on real data, travel plug-in on real data	Stretch
Document vault, persona, payments, iOS	No

Access progression: from notifications to deeper access

The app earns deeper access in steps instead of asking for everything on day one.

- **Phase 1 (POC): notification surfaces only.** The app works from three surfaces that all arrive as notifications from apps the user picks: ordinary app notifications (bank, biller, shopping, travel apps), the notification snippets the email app shows for new mail (for example Gmail's sender and subject preview), and the notification snippets the SMS app shows for new texts (bank and biller alerts, business senders only). These are processed on the phone and the text is discarded. No inbox is opened, and no Gmail or SMS permission is requested.
- **Phase 2 (after the pipeline is proven and users trust the app): opt-in deeper access.** The app offers two separate opt-ins, **Gmail (read-only) first, then SMS inbox access.** Gmail goes first because it is expected to be the easier permission for users to turn on: Google sign-in is already familiar, the scope is read-only, and the consent screen is Google's own. These opt-ins let it read the full email or message behind a notification, fill in details a snippet cuts off, and look back at history (for example, finding a bill that arrived before the app was installed). Each one is offered only where it adds something the snippet couldn't, for example "This bill notification was cut short. Allow Gmail access to read the full statement?"
- **When to move to Phase 2:** the accuracy report shows the notification pipeline meets its targets, the gaps that only fuller content can close have been measured (section 5.4), users are keeping the app and granting notification access, and the platform steps are done (Google Play's SMS permission declaration and Google's Gmail verification and security assessment, section 5.1). The exact thresholds are an open decision.
- **Where Phase 2 sits in the build: last, and not blocking.** The POC ships without Gmail or SMS inbox access. Those sources only raise how many items the app catches (fuller text, history, fewer missed bills); nothing in the POC depends on them. Phase 2 work starts only after the four-week base is done, in up to two optional extra weeks (section 4).
- **The privacy promise doesn't change.** Each new permission is its own consent scope and a new consent version, recorded in the ledger (section 11) with what was shown. Processing stays on-device first, and raw text is never stored or sent (section 10). Nothing is ever sold (section 14). Users can revoke Gmail, SMS inbox, or notification access at any time from the privacy dashboard, and the app falls back to the surfaces that remain.

3. Onboarding: demo first, permissions later

The app asks for some of the most sensitive access a phone can grant. People give that access more readily after they have seen what they get in return. So the first run shows the whole product working on sample data, and access is requested only after the user decides to try it on their own phone.

Step by step

Step	What the user sees	What the app reads	Consent / ledger
1. Welcome	One screen: the tagline, "See how it works with sample data", and a "Skip to setup" link	Nothing	None (no personal data is processed)
2. Demo journey	A realistic Today / Upcoming feed built from bundled sample data: an electricity bill due in 3 days, a SIP debit tomorrow, a bus trip tonight, a parcel out for delivery, a dentist appointment, and one festival sale item showing "4 teasers merged". The user can tap any item to see "why this is here" (the sample messages it came from, with numbers masked), mark a bill paid and watch the follow-up disappear, and open the privacy dashboard with sample activity	Nothing. The sample data ships in the APK	None
3. "Try it on your phone"	A short summary of what happens next: sign in, pick the apps whose notifications the app may read, then turn on notification access. It states plainly that no SMS inbox or email is read	Nothing	None
4. Account	Sign-in and the plain-language privacy notice	Nothing yet	Gate 1 event in the ledger (section 11)
5. Notification access	The notifications explainer (what is read, what is never stored, how to revoke), then the allowlist screen where the user picks apps (section 5.2), then the system settings page to enable access	Only notifications from allowlisted apps, from this moment on	Gate 2 event for notifications, plus one allowlist event per app, with the exact explainer text
6. First real result	Notifications give no history, so the app says so plainly: "We'll pick up bills as new notifications arrive. Add bills you already know about?" with a quick-add form. The first detected item is highlighted ("Found from your bank's SMS notification", numbers masked). Sample data is cleared completely and never mixed with real data	Allowlisted notifications	Audit entries
7. More apps, later sources	The user can add or remove allowlisted apps at any time from the privacy dashboard. In later phases, SMS inbox and Gmail are offered only when they add value, for example "Your electricity bill also arrives by email. Add Gmail?"	One more app or source per grant	New ledger event each time

Rules for demo mode

- Every sample screen carries a visible **Sample** badge and a banner: "This is sample data. Nothing from your phone has been read."
- Demo mode makes no network calls for data and needs no account. Sample data is fixture JSON in the app, reused as UI test fixtures.
- Demo mode can be reopened from Settings ("Show me the sample again") without touching real data.
- The demo shows verticals that the POC doesn't yet process for real (travel, delivery, appointments). Those cards are labelled "Coming soon on your data", so the demo never promises more than the app does.

This flow puts the trust elements of section 12 in front of the user in order: show value, explain, ask for one thing, then prove it with a real result.

4. POC plan and timeline

Repository. The code lives in a private GitHub repo named `orbit-one` on the `techhub-startup` account. It starts empty with just a README, and the week 1 build runs against it.

Honest timeline. The original two-week plan was already tight for finance alone. This version adds demo mode, a labelled test set with an accuracy report, on-device models, a per-app allowlist, basic Free/Pro gating, vault basics, and listener-reliability work.

Decision (26 Sep 2026): the committed base is four weeks of full-time work (20 working days). It can be extended by one or two weeks only for Phase 2 work (Gmail first, then SMS inbox), which comes last and does not block the POC. A three-week cut and a two-week minimum are listed after the table, along with what each one gives up.

What the POC must prove (this is also what makes it a strong senior-role portfolio piece; see section 15)

1. A working demo: the full journey on sample data, plus the real finance flow end to end.
2. A feasible data pipeline with **measured accuracy** on a labelled set of real, masked Indian messages.
3. A defensible privacy story: on-device processing, one-source-at-a-time consent, a versioned ledger, and an audit trail that proves it.

Week 1: app shell, demo mode, permission flow, consent ledger

Day	Goal	Done when
1	Repo setup: monorepo (<code>android/</code> , <code>backend/</code> , <code>infra/</code> , <code>docs/</code>), Gradle, detekt/ktlint, GitHub Actions build + test. App shell: Compose navigation, theme, and empty screens for feed, item detail, privacy dashboard, and settings	CI green; the app opens and moves between empty screens
2	Demo mode: life feed, item detail with "why this is here", dedup and paid-match animations, and a sample privacy dashboard, all from bundled sample data with Sample badges	The full journey in section 3 works in airplane mode with no account and no permissions
3	Auth decision (ADR), sign-in on Android; thin backend (section 6.4) validates JWT; Terraform deploys backend + DB	Signed-in user hits a protected <code>/me</code> endpoint in the cloud
4	Notification-listener permission flow: explainer screen, deep link to the system notification-access page, detection of granted/revoked state on every launch, and the per-app allowlist screen	A user can grant, see, and revoke notification access and pick allowlisted apps; the app reflects the real system state
5	Consent ledger (section 11): append-only events with version, timestamp, and what was shown, stored locally and synced to the server; audit log table. Listener wired to the allowlist with the SMS-app rule (business senders only, section 5.2) and the on-device finance pre-filter	Every grant and revoke writes a v1.0 ledger event on the server; allowlisted finance notifications appear in a debug list and everything else is dropped before parsing

Also in week 1: a synthetic notification generator. A small tool (a script plus a debug screen that posts fake notifications) generates varied test notifications: Hinglish text, grouped and truncated alerts, odd date and amount formats, promotional noise, and the same event arriving from several apps. It should produce about 200 samples, so extraction and dedup can be stress-tested from week 2 without waiting on real users, and they are kept separate from the real, masked labelled set so the accuracy report always says which set a number comes from.

Week 2: on-device extraction, tiny model, dedup, first real bill

Day	Goal	Done when
6	<code>LifeItem</code> schema and the finance plug-in (section 8.3); promotional filter rules (section 8.2); rules extractor for bank debits, SIP confirmations, premium notices, and utility/telecom bills, with unit tests; masked-export tool for collecting samples from consenting testers	Rules extract type, payee, amount, and due date from a fixture set
7	First 100 labelled samples and an eval script; iterate on the rules	Eval script prints per-field accuracy for the finance plug-in
8	LiteRT tiny model: MediaPipe Text Classifier (bill / payment / sale / other) behind the <code>TextClassifier</code> port, used only when rules are inconclusive (section 8.8)	Inconclusive items get a label on device in the background; the model file can be swapped without code changes

Day	Goal	Done when
9	Dedup of repeated events: event keys plus EmbeddingGemma similarity via the MediaPipe Text Embedder; merge rules for reworded repeats, date updates, and expiry (section 8.4), tested on sale and festival teasers and on bank-SMS plus biller-app pairs	Five reworded teasers for one sale become one upcoming item; the bank and biller notifications for one bill merge into one item
10	First real bill-due detection end to end: life feed on real data (Room), local reminders and follow-ups with WorkManager, payment matching (a later "paid/debited" notification closes the item), manual mark-paid and quick-add	A real bill notification becomes an item, the reminder fires before the due date, and the payment notification marks it paid

Week 3: cloud fallback, sync, Free/Pro gating, vault basics

Day	Goal	Done when
11	<code>LlmProvider</code> port with <code>OnDeviceProvider</code> (Gemini Nano Prompt API where available, LiteRT-LM otherwise) and <code>CloudProvider</code> (opt-in, paid tier); masking gate that blocks any snippet with an unmasked pattern	A hard case is resolved on device, and via cloud only when cloud-AI consent is on
12	Cloud fallback hardening: masking unit tests on real Indian formats, a daily call cap per user, audit entries for every cloud call, and a consent re-check before each call	Masking tests pass; turning cloud AI off stops calls immediately; every call appears in the audit log
13	Server sync of structured <code>LifeItem</code> s (never raw text) with RLS; simple conflict rule (latest update wins); export and delete-my-data covering device and server	A reinstall restores items after sign-in; delete-my-data wipes both sides and writes an audit entry
14	Free/Pro gating: entitlement flag served by the backend, gates for history window, tracked-bill limit, and vault (section 14); no real billing yet, a test toggle only. Privacy features are never gated	Switching the test entitlement changes the gated features and leaves privacy controls untouched
15	Vault basics: add a PDF or photo to encrypted on-device storage, tag it with a type and a manual expiry or renewal date, and get a reminder. AI clause extraction stays in P1	A warranty document added with an expiry date produces a reminder; files never leave the device

Week 4: reliability, edge cases, polish, end-to-end demo

Day	Goal	Done when
16	Listener reliability: detect disconnects and request a rebind, a "listener stopped" banner, OEM battery-optimisation guidance; test on at least two mid-range phones from different brands	Killing the listener shows the banner and recovers; results per phone are written down
17	Edge cases: grouped and truncated notifications, OTP redaction, silenced apps, date formats ("DD/MM", "tomorrow", "by 5th"), amounts written as ₹, Rs, or INR, Hinglish text, and duplicates across the SMS app and biller apps	Each case has a fixture and a test; grouped notifications don't create junk items
18	Battery and performance measurement on the mid-range phones (Android Studio profilers, Battery Historian); security pass (section 9 checklist, RLS tests, masking tests, logs scrubbed)	Battery and latency numbers recorded in <code>docs/</code> ; checklist ticked
19	Accuracy report on at least 200 labelled samples (per-field accuracy, dedup precision and recall); privacy dashboard polish (grants, allowlist, versions, revoke, activity counts)	Report committed to <code>docs/</code> ; the dashboard reads entirely from the ledger and audit log
20	End-to-end demo and polish: README with architecture diagram, ADRs, 2-minute demo video (demo mode, then the real flow from bill notification to paid), release APK	Portfolio-ready repo and demo

Also in week 4: the "first real catch" moment. Within the first minute after a new user grants notification access, the app should show one real item it genuinely caught (a bill, a reminder, or a delivery), so trust comes from their own data and not only from the sample demo. It does this by scanning the notifications already in the shade from allowlisted apps as soon as the listener connects, and if none qualify, it says so plainly and offers quick-add instead of showing a fake item.

What the fourth week buys over the three-week cut. The three-week version drops three things. It drops the **cloud AI fallback** and keeps on-device only, so hard cases on phones without an on-device LLM go to the user to confirm. It drops **server sync**, keeping only the ledger and audit log on the server, so there is no backup or restore and less backend work to show. And it drops most **reliability testing**, keeping only disconnect detection and the banner, so the plan's highest-rated risk (phone makers' battery managers stopping the listener) goes unmeasured. The three-week version also moves Free/Pro gating and vault basics to stretch goals. The fourth week turns those gaps into measured results, which is what makes the demo defensible in a senior-level review.

If it must fit in two weeks, cut everything above plus the Free/Pro gating, the vault, and the promo and sale dedup (keep bill dedup only). Keep demo mode, consent and the ledger, the notification listener with the allowlist, finance extraction, reminders and payment matching, and the accuracy report, because those three proofs are the point of the POC.

Optional weeks 5–6: Phase 2 (only after the base is done). These weeks are used only if Phase 2 needs them, and the POC counts as shipped at the end of week 4 either way. Work goes in this order:

1. **Gmail (read-only) first:** the Google sign-in scope request as its own consent scope and ledger version, fetching the full email behind a truncated notification, merging it into the existing dedup, and adding the Gmail cases to the labelled set and accuracy report. This runs in Google's testing mode with a small list of test users until the verification and security assessment in section 5.1 are done.
2. **SMS inbox second, if time allows:** reading the full text and history of business-sender SMS only, as a separate consent scope, with the same accuracy measurement. It is tested through direct APK or internal testing until the Google Play SMS permission declaration is approved.

Offering either one to the public still waits for the Phase 2 trigger in section 2.

Stretch if ahead: sales and events as an opt-in category on real data (section 8.4); a travel-booking plug-in on real confirmations, to show that a new vertical is one plug-in; a basic persona card (on-time payment rate).

5. Honest build risks

These risks are listed early because they decide what can ship publicly, not just what can be built.

5.1 Risk register

Risk	What it means	POC approach	Before any public launch
Google Play SMS restrictions (later phase)	Play restricts <code>READ_SMS</code> to default SMS apps and a short list of approved exceptions	Not applicable: the POC doesn't read the SMS inbox; bank SMS are seen as notifications	Before the SMS-inbox phase: get the Play permission declaration approved, or stay notification-only
Gmail restricted scope (later phase)	Gmail read scopes are "restricted" in Google's OAuth policy. Public use needs Google's verification and a paid third-party security assessment , which is a real cost and time item	Not applicable: no Gmail in the POC	Before the Gmail phase: budget time and money for verification and the assessment (testing mode with a few test users is fine for experiments)
Notification-access scrutiny	Android shows a strong system warning when a user enables notification access, because the app could read all notifications, including personal messages. Play's user-data policy requires prominent disclosure before access to personal and sensitive data, so expect close review of an app that reads notifications	Per-app allowlist, explainer before the system screen, deep-link to settings, no pressure prompts	Play Data safety form and privacy policy matching sections 10.2 and 10.3; a clear in-app disclosure screen
Notification-only limits	No history, truncated or grouped text, Android redaction of sensitive content, silenced apps, and OEM battery managers stopping the listener (section 5.4)	Quick-add, reliability work on day 16, measure the gaps in the labelled set	Decide from measured coverage when the SMS-inbox and Gmail phases are worth their cost
Life-wide privacy	"Any notification" includes personal chats and other private content	Allowlist-only processing (section 5.2)	Same, plus an external privacy review
Data pipeline accuracy	If extraction or dedup is wrong, the app misses bills or shows duplicates, and trust is lost faster than it was earned	Labelled test set and accuracy report in the POC (section 5.3)	Accuracy targets met per vertical before that vertical ships
On-device AI coverage	Most phones sold in India are not on the Gemini Nano device list (section 8.6)	Rules + tiny on-device models must work alone; LLM tiers are a bonus	Test on real low- and mid-range phones
Regulation	The DPDP Rules' main obligations start in the 18-month phase (commentators compute May 2027); a cloud AI provider is a processor, usually outside India	Consent ledger, masking, audit from day one (sections 9 and 11)	Legal review; re-check commencement and any transfer orders
iOS	Apple doesn't allow SMS or notification reading as done on Android	Android only	iOS would start with Gmail and manual entry (section 11)

Why notifications first. This path sidesteps the two biggest platform hurdles for the POC: Play's SMS restriction and the Gmail restricted-scope security review. Both remain real risks for the later phases that add those sources.

5.2 Life-wide means personal content: the allowlist rule

Reading "any notification" technically means the app is shown personal chats, OTPs, and other private content. The design keeps this honest:

- **Per-app allowlist, chosen by the user.** The notification listener processes only apps the user has explicitly allowed. Suggestions (known banks, billers, travel and shopping apps) are shown but not pre-selected, because consent should be a clear affirmative action.
- **Non-allowlisted content is dropped immediately, on device.** The listener checks the package name first and returns without parsing the text. Nothing is stored, logged (beyond an optional count), or sent anywhere.
- **Personal conversations are never stored**, even from allowlisted apps. Chat and social apps are not suggested for the allowlist, and the classifier drops personal and OTP content before extraction.

- **The SMS app needs a special rule.** Bank and UPI SMS arrive as notifications from the phone's SMS app, but so do personal texts. If the user allowlists the SMS app, the app processes only notifications whose sender appears to be a business (the alphanumeric sender IDs businesses use) and drops anything from a phone number or a saved contact before parsing the text (design recommendation; test it on the labelled set).
- **Building the allowlist picker without extra permissions.** The picker lists apps whose notifications have been seen (package names only) plus a declared list of known apps. This avoids the Play-restricted permission to query all installed apps (design recommendation).
- Each allowlist change is recorded in the consent ledger under the notifications scope (section 11).

5.3 The data pipeline is the main de-risk item

Everything else in this plan is known engineering. Whether the app extracts the right items from real Indian messages, and merges duplicates correctly, is the open question. So the POC measures it:

1. **Labelled test set.** Collect real notification samples from consenting testers, starting with Pavan's own phone (plus SMS and email samples before those phases). Mask them on the device before export (names, numbers, account references replaced with placeholders). Label kind, counterparty, dates, amount, and a duplicate-group id. Aim for at least 200 samples in the POC, including Hinglish, Indian bank and biller formats, UPI messages, grouped and truncated notifications, and repeated sale teasers. Recording which bills a tester actually had that month also measures what notifications miss.
2. **Measure before scaling.** Report per-field accuracy and dedup precision/recall (section 8.9). Set a target per vertical (the exact numbers are an open decision) and don't add a vertical, or launch one, until it meets that target.
3. **Re-run on every change** to rules, prompts, models, or thresholds, in CI.

Longer-term regulated finance data routes. Parsing messages is the practical start, but India has regulated, consent-based routes that could replace or check parsing for finance later:

- **RBI Account Aggregator framework.** RBI-licensed NBFC-AAs move financial information from providers (banks, insurers, AMCs and others) to users only with the customer's explicit, revocable consent. A data consumer (FIU) must be an entity regulated by a financial sector regulator, so this app would need a regulated partner or its own licence. The Department of Financial Services reports 179 FIPs and 989 FIUs live as of 31 Mar 2026.
- **Bharat BillPay, now branded Bharat Connect.** NPCI Bharat BillPay Ltd renamed the Bharat Bill Payment System "Bharat Connect" in August 2024. It is India's interoperable bill payment system. Integrating through a participating bank or operating unit could give authoritative bill amounts and due dates for supported billers, plus in-app payment (P2).

5.4 Notification-only trade-offs (stated honestly)

Limitation	What it means	Mitigation
No history	The listener sees only notifications posted after access is granted, so bills from last week are invisible	Quick-add for known bills at onboarding; say plainly that items appear as new notifications arrive; the Phase 2 Gmail and SMS inbox opt-ins can add history later
Truncated or grouped text	Some apps post short previews or summaries such as "3 new messages", so the amount or date may be missing	Extract what's there, mark missing fields, and ask the user or wait for a fuller notification; measure how often it happens
Sensitive content redacted by Android	Recent Android versions hide sensitive content such as one-time passwords from notification listeners	Nothing to fix; the app drops OTPs anyway
Silenced apps	If the user turns off notifications for an app in system settings, the listener doesn't see them	Show which allowlisted apps have been quiet for a long time; the user decides
OEM battery managers	Some manufacturers' battery optimisers can stop the listener, and then nothing arrives	Detect disconnects, request a rebind, show a "listener stopped" banner, and guide the user to the battery-optimisation setting for their phone

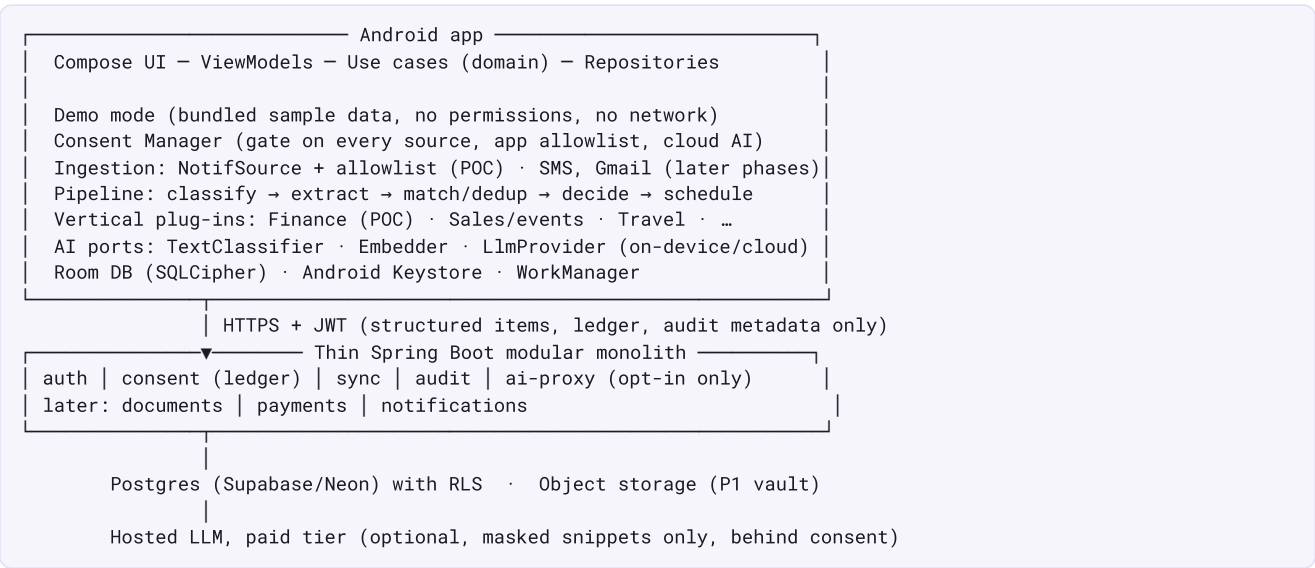
In return: real-time signals, no inbox collection, a permission the user can see and control per app, and a POC that avoids the SMS and Gmail platform reviews. The on-device agent processes and discards notification content locally, and nothing raw leaves the phone.

5.5 Key risks to watch

Dedup is the main risk to prove in week 2 (day 9). One real-world event, such as an Amazon sale, Flipkart's Big Billion Days, or Amazon's Great Indian Festival, can arrive as five separate notifications from different apps, with different wording and sometimes shifting dates. Merging those into one item without also merging two genuinely different events is harder than it looks (section 8.4). If the day 9 test isn't passing on the labelled set by the end of week 2, treat it as a blocker rather than leaving it for later.

6. Architecture

6.1 High-level shape



Clean-architecture rule: domain and use-case layers have no Android, Spring, or vendor imports. Vendors live behind interfaces (ports) and are wired in at the edges (adapters). This is what makes the free-to-paid swaps in section 13 and the model swaps in section 8.7 cheap.

6.2 Data flow (one item from signal to "done")

1. The user has granted notification access and allowlisted apps (for example their SMS app, bank app, and electricity-board app). The consent events are stored locally and on the backend with scope, timestamp, and consent-text version.
2. The notification listener receives a signal. Notifications from non-allowlisted apps are dropped at the package check, before any text is read; for the SMS app, only business senders go further (section 5.2). In later phases, SMS-inbox and Gmail adapters feed the same pipeline.
3. The on-device classifier labels it (bill, SIP, premium, payment confirmation, sale, booking, delivery, OTP, personal, other). OTPs, personal content, and categories the user hasn't turned on are dropped and never stored.
4. The matching vertical plug-in extracts a typed `LifeItem`: rules first, then on-device models (section 8).
5. If the item is still unresolved **and** the user opted into cloud AI, a masked snippet goes through the backend's `ai-proxy` to a paid-tier hosted model. Otherwise the item waits on device for the user to confirm it.
6. The matcher checks for an existing item (same bill from another source, same sale from another teaser) and decides: create, update, merge, or ignore.
7. The structured `LifeItem` is saved in encrypted Room and, if sync is on, sent to the backend. The raw notification text has already been discarded on the device; it is never stored or sent.
8. The scheduler creates or updates reminders. A follow-up job watches for the "done" signal for that vertical ("paid / debited" for a bill, "delivered" for a parcel) and closes the item, or nudges the user.
9. Every access, AI call, allowlist change, and consent change writes an audit entry.

6.3 Where the AI runs

Step	Location	Why
Allowlist check and dropping personal/OTP content	On device, code and rules	Private content never leaves the phone and is never parsed beyond what's needed to drop it
Classification ("is this a bill / a sale / a booking?")	On device: rules, then a tiny classifier or embeddings	Works on every phone, in the background, at no cost per message
Field extraction (common Indian formats)	On device: rules and templates; on-device LLM where supported	Cheap, private, works offline

Step	Location	Why
Dedup and matching	On device: keys, fuzzy match, embeddings; LLM tiebreak only when ambiguous	Needs the user's existing items, which live on the device
Hard cases	Backend <code>ai-proxy</code> → hosted model, masked input only	Only after cloud-AI consent (Gate 3)
Document extraction (P1)	Backend, encrypted in transit and at rest, processed then discarded	Documents are too large for device models; explicit per-document consent
Persona (P1)	Backend, over structured history only	No raw text needed

6.4 Recommendation: a thin backend for the POC

Because classification, extraction, dedup, and reminders all run on the phone, the POC backend only needs to do what a phone can't do alone:

- **auth** : validate the identity provider's JWT and expose `/me` .
- **consent** : hold the authoritative, append-only consent ledger (section 11).
- **audit** : store audit metadata (never message content).
- **sync** : store structured `LifeItem` s for backup and history (sensitive columns encrypted at the application level). This is dropped in the three-week and two-week cuts.
- **ai-proxy** : only for users who turned on cloud AI. It forwards masked snippets to the hosted model, so the provider key never ships in the app, and it logs token counts for cost tracking.

Deferred modules: `documents` (vault), `payments` , and `notifications` (server-side push; the POC uses local `WorkManager` reminders).

It stays a modular monolith with internal interfaces, so any module can be extracted later without rewrites. A thin backend also shrinks the attack surface and the free-tier footprint, and supports the privacy story: the server never holds raw messages.

6.5 Consent gates

- **Gate 0, demo**: no consent needed, because nothing is read (section 3).
- **Gate 1, account**: sign-up and plain-language privacy notice (English first, Hindi later).
- **Gate 2, per source, one at a time**: in the POC the only source is notification access. SMS inbox and Gmail come later as separate grants, each with its own explainer. Revocable any time; revoking stops ingestion and offers deletion of derived data.
- **Gate 2a, per-app allowlist**: for notifications, the user chooses each app (section 5.2).
- **Gate 3, cloud AI**: off by default. The app works, with less accuracy on hard cases, without it.
- **Gate 4, per category or vertical**: sales and events, and each later vertical (travel, deliveries, appointments, renewals), are separate opt-in purposes.
- **Gate 5, per action (P1)**: AI-proposed reminders from documents stay suggestions until the user accepts each one.
- **Gate 6, payments (P2)**: explicit confirmation on every payment; no stored card data.

Android platform constraints (notification access in the POC; SMS policy and Gmail restricted scope for later phases) are covered in sections 5.1, 5.4, and 10.4.

6.6 Offline behaviour: local-first with a sync queue

Extraction, dedup, reminders, and the feedback loop (section 8.5) all run on the device, so the core experience never depends on signal. Structured items, along with consent and audit events, wait in a local sync queue (Room plus a `WorkManager` job that runs whenever the network is available), which drains automatically when connectivity returns, with no user action. Reminders scheduled while offline fire from local storage at the right time, and their status (fired, snoozed, paid) syncs later. The cloud AI fallback simply doesn't run offline: on-device rules and models handle everything, and anything they can't decide is saved with a low-confidence flag for later review instead of failing. When offline, the user sees a subtle "syncing" indicator, never an error.

7. Stack and why

Layer	Choice	Career value	Fit for this app	Cost
Language	Kotlin everywhere (Android + backend)	One language across the stack; strong signal for Android + backend roles	Shared DTOs and domain models; coroutines on both sides	Free
Android UI	Jetpack Compose	Current Android standard; expected in interviews	Fast iteration on the feed, demo mode, and consent screens	Free
Local data	Room (encrypted via SQLCipher)	Standard offline-first pattern	Items, embeddings, and the local ledger copy stay on device, encrypted	Free
Networking	Retrofit + OkHttp (+ Kotlinx Serialization)	Industry standard	Typed API client to the thin backend	Free
Background work	WorkManager	Core Android skill	Reminders, follow-ups, purges, listener health checks, heavy AI jobs under charging/idle constraints	Free
On-device AI	LiteRT via MediaPipe Tasks (classifier, EmbeddingGemma embeddings); ML Kit GenAI Prompt API (Gemini Nano) where supported; LiteRT-LM (small Gemma) elsewhere	Current on-device AI skills with a clean port design	All processing on the phone by default (section 8)	Free per message
Backend	Spring Boot (Kotlin), thin (section 6.4)	Leverages 3+ years of Spring strength; shows depth, not a new framework	Mature security, validation, JPA/JDBC, Actuator	Free
Backend (upgrade)	Ktor	Shows range later; lighter and faster cold starts	Worth it if serverless cold starts hurt	Free
Database	Postgres on Supabase or Neon (free tier)	Postgres + row-level security is a sought-after skill	RLS enforces per-user isolation at the DB layer	Free tier, paid plans when needed
DB access	Exposed or jOOQ	Type-safe SQL; shows you care about queries, not just ORM magic	Explicit control for RLS session variables and audit queries	Free (jOOQ open-source edition for Postgres)
Auth	Firebase Auth or Auth0 (decide in week 1)	Real-world identity provider integration, JWT validation	Google sign-in pairs naturally with the later Gmail phase	Free tier
Cloud AI (opt-in)	LlmProvider port with one paid-tier hosted-model adapter	Clean abstraction and cost-aware AI design	Swap providers without touching domain code	Pay per token; small at fallback volumes (section 8.6)
CI/CD	GitHub Actions	Standard	Build, test, lint, eval run, deploy on merge	Free for public repos; free minutes for private
Infra as code	Terraform	Senior-level signal	Reproducible env; same code for free and paid tiers	Free
Hosting	Container on Google Cloud Run (scales to zero) or AWS using free credits	Cloud-native deployment on familiar AWS or a second cloud	One Docker image, portable between providers	Free tier at POC traffic
Architecture	Modular monolith: <code>auth</code> , <code>consent</code> , <code>sync</code> , <code>audit</code> , <code>ai-proxy</code> now; <code>documents</code> , <code>payments</code> , <code>notifications</code> later	Shows judgement: right-sized design, not premature microservices	Each module can be extracted into a service later without rewrites	Free

Decision to log in week 1: Firebase Auth vs Auth0. Firebase is simpler with Google sign-in and Android; Auth0 is more portable and enterprise-looking. Pick one and record it in an ADR (architecture decision record).

8. AI agent design (explained from scratch)

Pavan wants most of the project's effort to go into this layer. This section explains how it works from first principles, for a backend engineer new to LLMs. It generalises the design from finance to any part of life through vertical plug-ins, and records what on-device and cloud AI offer as of September 2026. Facts about products, prices, and law are cited in 8.11 with the date they were checked.

8.1 Terms in plain words

Term	What it means here
Model	A file of learned numbers (weights) plus code that turns an input into an output. Running it is called inference .
LLM (large language model)	A model that reads text and produces text one small piece at a time, predicting the most likely next piece given everything before it. It has no memory between calls and no access to your database unless you give it data in the request.
Token	The unit an LLM reads and writes: a word, part of a word, or a symbol. Cloud providers bill per million tokens, separately for input and output.
Prompt / system instruction	The text sent to the model. The system instruction is the fixed part ("You extract booking details from Indian messages..."); the rest is the message being processed.
Context window	The maximum number of tokens one call can include. Gemini Nano through ML Kit accepts under 4,000 input tokens per call.
Structured output	Asking the model to answer in a fixed shape (a JSON schema or an annotated Kotlin data class) instead of free text, so code can parse it and reject anything that doesn't fit.
Tool (function) calling	You describe functions to the model (name, parameters, purpose). Instead of answering in prose, the model can reply "call <code>searchItems(counterparty="amazon")</code> ". Your code runs the function and sends the result back. The model never executes anything itself.
Agent	A program that calls an LLM in a loop: the model picks a tool, code runs it, the result goes back to the model, and this repeats until the model gives a final answer.
Embedding	A fixed-length list of numbers (a vector) produced by an embedding model. Texts that mean similar things get vectors that point in similar directions, even if the wording differs.
Cosine similarity	A number from -1 to 1 that measures how closely two embeddings point the same way. Higher means more similar in meaning. Dedup uses it with a threshold.
Classifier	A small model that outputs a label and a confidence ("sale: 0.93"). It is much cheaper to run than an LLM because it does one pass over the input instead of generating text.
Quantization	Storing weights with fewer bits (for example 8-bit or 4-bit instead of 16-bit) so the model uses less disk and RAM. Quality may drop slightly.
Hallucination	The model states something that isn't in the input, such as an invented date. Schemas, validation against the source text, and confidence checks catch most of these.
Eval set	A labelled collection of real examples used to measure accuracy before and after every change. For LLM features, this is the equivalent of a unit-test suite.
Precision / recall	Precision: of the items the system flagged (as duplicates, as bills), how many were right. Recall: of the items it should have flagged, how many it caught.

8.2 What "agent" means in this app

Two ways to wire an LLM in

- **Autonomous agent loop.** The model receives the message plus tools such as `searchItems`, `createItem`, `updateItem`, and `scheduleReminder`, and decides what to call and in what order. It is flexible, but every step is another model call, the path differs between runs, it's hard to test, and the model gets to trigger writes.
- **Fixed pipeline with LLM steps.** Ordinary Kotlin code runs the same stages every time. The LLM is called only at specific points ("extract these fields into this schema", "are these two items the same? yes/no"). Code makes every decision that changes data.

Recommendation for the POC: a fixed pipeline with LLM steps. Reasons:

- **Cost:** one bounded call (or none) per message.

- *Predictability*: the same input takes the same path, so bugs can be reproduced.
- *Testability*: each stage has typed inputs and outputs, and the LLM stage can be faked in tests.
- *Privacy*: only the extraction or tiebreak step sees text, and only the minimum snippet. Write tools are never exposed to the model.
- *Device limits*: on-device models have small context windows and quotas (8.6), which suit single short calls.

Tool calling is still worth learning. The dedup tiebreaker can be written as a single tool-style call, and a later opt-in "ask the assistant" feature could use a real agent loop over structured items only.

Pipeline for one incoming signal

#	Stage	What happens	Typical implementation
1	Ingest	The notification listener hands over app package, title, text, and timestamp (later, SMS and Gmail adapters too). Consent and the app allowlist are checked first; non-allowlisted notifications and personal SMS stop here	Kotlin, no AI
2	Classify	Run the promotional filter first (below), then label the vertical and kind (bill, SIP, premium, payment confirmation, sale, booking, delivery, appointment, OTP, personal, other). Promotions, OTPs, personal content, and categories the user hasn't enabled are dropped	Rules and sender lists first, then a tiny on-device classifier or embedding nearest-neighbour; LLM only if still unsure
3	Extract	The matching vertical plug-in fills its typed schema, validates it (date parses, amount positive, counterparty known or normalised), and maps it to a <code>LifeItem</code>	Regex and templates for known senders; ML Kit Entity Extraction for dates and money; on-device LLM with structured output when rules fail
4	Match / dedup	Compute the plug-in's dedup key and look for an existing item with the same key, a fuzzy match, or high embedding similarity (8.4)	Kotlin + Room queries + embeddings
5	Decide action	<code>CREATE</code> , <code>UPDATE</code> (for example a date change), <code>MERGE</code> (duplicate, added as evidence), or <code>IGNORE</code> . Code decides; an LLM yes/no is used only for ambiguous matches	Kotlin <code>when</code> over match results
6	Schedule	Create or update reminders using the plug-in's reminder policy; cancel reminders for merged, done, or expired items	WorkManager
7	Audit	Record source, stage outcomes, tier used (rules / on-device / cloud), and whether anything left the device. No raw text	Audit log (section 9)

Raw text is discarded after stage 5. What persists is the structured item, a short canonical summary, and its embedding.

Promotional filter (stage 2 rule)

Promotional and marketing notifications are recognised on the device and skipped, so they are never parsed as actionable items such as due dates, OTPs, deliveries, or bills. A "sale ends tonight" blast must never become a payment reminder.

Where it runs: at the start of stage 2, before any vertical plug-in sees the text. It is rules only, runs in the background, and costs nothing per message.

What it looks for (signals add up to a promo score; one keyword alone is not enough):

- *Marketing phrases*: "unsubscribe", "offer", "sale ends", "limited time", "flat X% off", "up to X% off", "use code", "cashback up to", "deal of the day", "hurry", "last chance", "shop now", "T&C apply", and similar discount blasts.
- *Unsubscribe and opt-out messages*: "you have been unsubscribed", "reply STOP to opt out", and subscription-preference confirmations.
- *Sender and channel metadata*: notifications posted with Android's `Notification.CATEGORY_PROMO` or on an app's promotions channel, and, for bank and business SMS shown by the SMS app, the sender header's category marker where one is present (TRAI's sender-header rules distinguish promotional from transactional and service senders; check the current format before relying on it).
- *Shape*: no amount tied to the user's account, no reference number, and no personal due date, but a discount percentage, a coupon code, or a link.

Guardrail against skipping real items: transactional signals override the promo score. An amount debited or credited, "due on" or "bill generated", masked account or card digits, "payment received", a booking or order reference, or an OTP pattern keeps the message in the pipeline even if it also contains "offer". Many bank alerts end with an advertising line. In that case, the promotional tail is

stripped and only the transactional part goes on to extraction. If the two scores are close, the tiny on-device classifier (bill / payment / sale / other) decides. It never guesses toward "promo" for a message from a known biller or bank without checking.

What happens to a promotion:

- If the user has turned on the opt-in Sales and events category, the message goes only to the Sales and events plug-in (8.4). There its dates become sale start and end times, never a due date.
- Otherwise it is dropped on the device immediately, and its text is never stored. The audit log records only a count ("14 promotions skipped today"), which the privacy dashboard can show.

User feedback: "This wasn't a promotion" sends the message back through extraction and stores the pattern as a local exception. "Hide promotions from this app" mutes that app's promotions. Both stay on the device.

How it's tested: the labelled set includes pure promotions, unsubscribe confirmations, and mixed messages (a real bank debit with an offer line attached). The accuracy report tracks two numbers: how many promotions slipped through as items, and, more importantly, how many real bills or payments were wrongly skipped. The target for wrongly skipped bills should be close to zero and is set as an open decision.

8.3 One schema for life: `LifeItem` and vertical plug-ins

The finance-only `ExtractedItem` from v1 becomes a vertical-agnostic `LifeItem`. Everything vertical-specific moves into plug-ins, so adding travel or deliveries later means adding one plug-in, not changing the pipeline.

```
enum class Vertical { FINANCE, SALES_EVENTS, TRAVEL, DELIVERY, APPOINTMENT, RENEWAL, OTHER }
enum class ItemStatus { UPCOMING, LIVE, DONE, CANCELLED, EXPIRED }

data class LifeItem(
    val vertical: Vertical,
    val kind: String,                // plug-in defined, e.g. "electricity_bill", "sale", "bus_trip"
    val counterparty: String?,       // normalised biller / merchant / operator, e.g. "amazon", "redbus"
    val title: String,               // short canonical summary, e.g. "Amazon festival sale"
    val startsAt: ZonedDateTime?,    // due date, sale start, departure, delivery window, appointment
    val endsAt: ZonedDateTime?,
    val status: ItemStatus,
    val amountPaise: Long?,          // money as integer paise, never floating point
    val referenceMasked: String?,    // last 4 of an account or booking reference only
    val attributesJson: String,      // plug-in specific fields, validated by the plug-in
    val dedupKey: String,
    val confidence: Float,           // from rules (fixed) or model
    val source: ExtractorSource      // RULES, ON_DEVICE_MODEL, ON_DEVICE_LLM, CLOUD_LLM
)

interface VerticalPlugin {
    val vertical: Vertical
    val consentScope: String         // e.g. "vertical.finance", "category.sales_events"
    fun score(signal: RawSignal): Float // cheap rules: does this plug-in apply?
    fun extractWithRules(signal: RawSignal): LifeItem?
    val llmSchema: KClass<*>          // small typed class for LLM fallback extraction
    fun toLifeItem(llmResult: Any, signal: RawSignal): LifeItem?
    fun dedupKey(item: LifeItem): String
    fun merge(existing: LifeItem, incoming: LifeItem): LifeItem
    fun isDoneSignal(signal: RawSignal, item: LifeItem): Boolean
    fun reminders(item: LifeItem): List<ReminderSpec>
}
```

Each plug-in's `llmSchema` is a small typed Kotlin class that works with ML Kit's Structured Output API (annotated with `@Generable` / `@Guide`; strings, numbers, booleans, lists, and nested classes are supported), with a JSON schema for a cloud model, and with rule-based code. The pipeline never cares which of these produced the result.

Plug-in	Example signals	Dedup key	"Done" signal	Reminders	Phase
Finance	POC: bank and UPI SMS shown as notifications by the SMS app, bank and biller app notifications, SIP debit and premium notices. Later: SMS inbox, e-statement emails	biller \ referenceLast4 \ billingCycle (amount and due date as tie-breakers)	"Paid / debited / successful" message, or manual mark-paid	Before due date; follow-up if unpaid	POC
Sales and events	Festival-sale teasers from shopping apps	merchant \ sale \ dateWindow	Event end date passes	Day before start, "live now"	Stretch (opt-in)
Travel	Bus, train, or flight booking confirmations (e.g. Redbus) by email or notification	operator \ bookingRefLast4 \ departureDate	Departure time passes, or a cancellation message	Day before, before departure; updates on reschedule	P1
Deliveries	"Shipped", "out for delivery", "delivered"	merchant \ orderRefLast4	"Delivered" message	Delivery day	P1
Appointments	Clinic, salon, or service booking confirmations	provider \ date \ time	Appointment time passes	Day before, an hour before	P1
Renewals	Insurance, subscription, and licence renewal notices	provider \ referenceLast4 \ renewalDate	Renewal confirmation	Weeks before, then days before	P1

8.4 Event detection and deduplication (the Amazon sale case)

What should happen

1. Day 1: a teaser arrives ("Big festival sale starts 8 Oct, get ready!"). The app creates an Upcoming item: *Amazon sale starts on 8 Oct*.
2. Days 2–10: more teasers arrive with different wording, emoji, and offers. The app recognises them as the same event and records each one as evidence on the existing item instead of creating new items.
3. One teaser says "starts 6 Oct" (the date moved). The app updates the item and shows "date changed".
4. "Starts tomorrow" and "Live now" update the status; "Ends tonight" sets the end date.
5. After the end date, the item expires and leaves the feed automatically.

How it works

- **Canonical dedup key** from the plug-in, for example `amazon|sale|2026-W41`. The counterparty is normalised from the SMS sender ID, notification app package, or email domain, plus a small alias table. The date window is a coarse bucket (a week, or "undated") so that a small date shift still lands on the same key. Relative dates ("tomorrow", "this Friday") are resolved against the message timestamp in IST before the key is built.
- **Matching ladder** (cheapest first; stop at the first confident answer): 1. Exact key match → same item. 2. Same counterparty and kind, with a date inside a tolerance window (or one side undated) → candidate. 3. For candidates, compare on-device embeddings of the canonical summaries (cosine similarity). Above a high threshold → duplicate. Below a low threshold → new. In between → ambiguous. 4. Ambiguous only: one small LLM call with both summaries ("same item? yes/no + reason", structured output). On-device if supported; otherwise cloud (only with cloud-AI consent, masked text); otherwise show both with a "possible duplicate" chip for the user to resolve. Tune both thresholds on the labelled set (8.9). Don't guess them.
- **Merge rules** (plain code, not the LLM): the latest explicit date wins, and the previous value is kept in the item's history. Relative phrases update status. A merge never creates a second reminder; an update reschedules the existing one.
- **Expiry**: an item expires after its end date, or after a default duration if none is known. Expired sale items and their embeddings are deleted after a short grace period.
- **User feedback (the personalisation loop)**: "Not a duplicate" splits the item and stores the pair as a negative example. "Ignore this app" removes it from the allowlist. "Not interested in sales" turns off the category. Feedback stays on the device and adjusts per-user thresholds and mute lists; no model training is needed for the POC.
- **Priorities across verticals**: money and time-critical items (a bill due tomorrow, a bus tonight) always outrank sales. Sale items sit in their own lane or filter.
- **The same ladder serves every vertical**. A bill's SMS reminder (seen as a notification) and the biller app's notification merge into one item with two pieces of evidence; in later phases, the email statement joins them. A bus booking's confirmation email and the operator app's reminder merge the same way. Detecting "done" (a payment confirmation, a "delivered" message) is the same

matcher pointed at completion signals.

8.5 Feedback loop: "That's not right"

Every extracted item (a bill, a payment, a reminder, a delivery) has a "That's not right" option. The user can simply flag it, or correct the category, date, or amount. OTPs are never extracted or stored (they are redacted), so they have nothing to correct. Corrections are stored on the device as structured fields only (the item's category, date, amount, and counterparty, before and after), never the raw notification text. They feed back in three ways: they add fixtures to the labelled set, they tune that user's rules and dedup thresholds (alongside the "Not a duplicate" feedback in section 8.4), and they show which extraction rules to fix next. If a correction is ever shared to improve the rules for everyone, sharing is opt-in, recorded in the consent ledger, and sends only the structured fields.

8.6 On-device vs cloud AI: what exists in September 2026

On-device options

Option	What it is	Device coverage	Key limits (from official docs)
Gemini Nano via ML Kit GenAI APIs (AICore)	Google's on-device model, shipped and updated by the Android system service AICore and shared by all apps. The Prompt API takes custom prompts and supports structured output.	Only devices on Google's published list. The Prompt API list is mostly flagships and upper mid-range: Pixel 9/10/11, Galaxy S26, Z Fold7/8, Z Flip8, OnePlus 13/15, Xiaomi 15/17, vivo X200/X300, several OPPO, POCO, realme, iQOO, and Honor models. Different devices run different Nano versions (nano-v2/v3/v4).	Prompt API is beta and Structured Output is alpha . Input must be under 4,000 tokens. A per-app inference quota applies, including a battery-use quota. Inference is allowed only while the app is in the foreground ; background use, including a foreground service, returns <code>BACKGROUND_USE_BLOCKED</code> . Not supported on unlocked bootloaders. Output can differ between Nano versions.
LiteRT-LM with open models (e.g. Gemma)	Google's open-source LLM runtime (the successor to the MediaPipe LLM Inference API, which is now maintenance-only). Kotlin API is stable. Supports CPU, GPU, and NPU, plus tool calling and multimodal models. You ship or download the model yourself.	Any Android device that has enough RAM and storage for the chosen model. No allow-list.	Model load (<code>engine.initialize()</code>) can take up to about 10 seconds. Memory to load Gemma 4 E2B is about 1.1 GB in the mobile build (0.84 GB text-only), 2.9 GB at Q4_0, and 5.7 GB at 8-bit. Gemma 3 1B NPU builds are 658 MB (4-bit) to 1,704 MB (8-bit).
llama.cpp and similar (MLC LLM, ExecuTorch, ONNX Runtime GenAI)	Other runtimes for open models in formats like GGUF. See 8.7.	Varies. llama.cpp's Android binding auto-detects CPU features and works on older, low-RAM devices.	You manage model files, updates, and quality.
On-device embeddings	EmbeddingGemma (308M parameters, 100+ languages, 2K-token input, under 200 MB RAM when quantized), run through the MediaPipe Text Embedder task, which supports it directly.	Broad, because it runs on CPU on ordinary phones.	Produces vectors, not text. Ideal for dedup and nearest-neighbour classification.
Classic on-device ML Kit	For example, Entity Extraction finds dates, money amounts, and payment-card numbers in text.	Broad (not tied to the GenAI device list)	Beta. Supported languages don't include Indian languages. Precision is favoured over recall.

What this means for Indian users. IDC reports that 83% of India's 2025 smartphone shipments cost under US\$400: 16% were under US\$100, 41% were US\$100–200, and 26% were US\$200–400. Most models on the Gemini Nano Prompt API list are flagship or upper-mid-range lines. My inference from these two sources: most target users will **not** have Gemini Nano, and a design that needs it would exclude the majority. The rules and embedding tier must work alone. Gemini Nano is a bonus where present, and LiteRT-LM with a small Gemma model is the portable middle option for phones with enough RAM.

Battery and latency. No official source publishes comparable battery figures for these runtimes. What the docs do show: AICore enforces a per-app battery quota; Google's MediaPipe LLM guide describes itself as optimised for high-end devices (Pixel 8, Galaxy S23 or later); and LiteRT-LM model loading takes seconds. In practice, run LLM work in batches, keep the model loaded only while needed, and measure on two or three real low- and mid-range phones before deciding.

Cloud models: how pricing works. Hosted models charge per million input tokens and per million output tokens. With "thinking" models, the thinking tokens are billed as output. Published paid-tier prices on Google's Gemini API pricing page (last updated 24 Sep 2026):

Model	Input / 1M tokens	Output / 1M tokens
Gemini 2.5 Flash-Lite	\$0.10	\$0.40
Gemini 3.1 Flash-Lite	\$0.25	\$1.50
Gemini 3.5 Flash-Lite	\$0.30	\$2.50

Batch or Flex modes cost about half, but they are asynchronous.

Worked estimate (method first, then one example). Monthly cost per user = cloud calls per user × (input tokens × input price + output tokens × output price) ÷ 1,000,000. Example assumptions: cloud is used only as a fallback, 60 calls a month, about 500 input tokens each (instructions plus a masked snippet), and about 100 output tokens (a small JSON). On Gemini 3.1 Flash-Lite that is 30,000 input tokens (US\$0.0075) plus 6,000 output tokens (US\$0.009), about **US\$0.017 per user per month**. On Gemini 2.5 Flash-Lite it's about US\$0.005. These are assumptions, not measurements; a life-wide app may send more signals per user than finance alone, so replace them with real counts from the audit log once the POC runs.

The catch with "free". The Gemini API free tier is free of charge. However, Google's terms say that for unpaid use, content may be used to improve Google's products and may be read by human reviewers, and they state: do not submit sensitive, confidential, or personal information. So **real user data must only go to a paid tier**, which is not used for product improvement and is covered by Google's data-processor terms. The free tier is fine for synthetic or fully anonymised test data during development. Paid-tier prompts are still logged for a limited period for abuse detection, and may be stored transiently in any country where Google has facilities.

Privacy and DPDP implications

Topic	On-device	Cloud
Where data goes	Stays on the phone. Nothing to disclose beyond local processing.	Goes to a processor (the model provider), usually outside India. For example, Gemini 3.5 Flash-Lite on Google Cloud lists ML processing only in US and EU multi-regions.
DPDP roles	The app is still the Data Fiduciary for what it derives and stores.	The provider is a Data Processor. The fiduciary stays responsible for security safeguards "including in respect of any processing undertaken... by a Data Processor" (DPDP Rules, Rule 6), which also names masking and obfuscation as safeguards.
Cross-border	Not applicable.	DPDP Rule 15 permits transfer outside India, subject to any government orders. Section 16 lets the government restrict specific countries. Both are in the group that comes into force 18 months after the Rules were published on 13 Nov 2025 (commentators compute this as May 2027). Sector rules, such as RBI rules for payment data, apply separately.
Consent and notice	Covered by the per-source consent.	Needs the separate cloud-AI consent (Gate 3) naming the processor. A new provider is a new consent version (sections 11 and 12).
Minimisation	Raw text is discarded after processing.	Send only masked snippets: mask account and card numbers, phone numbers, names, and addresses; keep counterparty, dates, and amounts. Log in the audit trail that redaction ran.

8.7 On-device runtime choice (future-proofing)

The question is which on-device library to build on for tasks like "is this a sale notification?" without locking the app to one library or one model.

Two different kinds of model

- A **tiny text classifier or embedding model** (tens to a few hundred million parameters, or smaller) does one pass over a short text and returns a label or a vector. It runs on the CPU of ordinary phones, works in the background, and uses little memory. EmbeddingGemma, for example, needs under 200 MB of RAM when quantized.
- A **~2B-parameter LLM** generates text token by token. It needs roughly 1–3 GB of RAM depending on the build, takes seconds to load, and in Gemini Nano's case is restricted to certain devices and to foreground use.
- For "is this a sale?" or "is this a booking?", the first kind is enough. The second kind is for extracting messy fields and for ambiguous tiebreaks.

Is "rules first, small model only when inconclusive" still right in 2026? Yes for this app, with one refinement. On-device LLMs have clearly improved: Gemini Nano now has a custom Prompt API with structured output, LiteRT-LM supports tool calling and NPUs, and Gemma 4 E2B has a 128K context window. But the constraints that matter here remain, according to the official docs:

- limited device coverage in India (8.6);
- ML Kit GenAI inference only in the foreground, while notifications arrive in the background;
- per-app quotas, including battery;
- load times of several seconds;
- output that differs between Nano versions.

The refinement is that the first tier should be **rules plus a tiny classifier or embedding model**, not rules alone. Embedding nearest-neighbour ("is this close to known sale teasers?") handles reworded messages that regex misses, on every device, at no marginal cost, and it matters more in a life-wide app where formats are far more varied than bank SMS. The LLM is reserved for what the cheap tiers can't settle. This is my recommendation based on these constraints. I did not find an official Google statement that prescribes this pattern.

Runtime comparison (facts from official docs as of Sep 2026; "not published" means I found no official figure)

Runtime	Best at	Runs well on	Battery	~2B model size (disk / RAM)	Swapping models later	iOS reuse path	Status
LiteRT (formerly TensorFlow Lite), incl. MediaPipe Tasks Text Classifier / Text Embedder	Tiny classifiers and embedders	All tiers (CPU by default; GPU/NPU optional). Also available as a Google Play services runtime	Not published; small for tiny models, since it does one pass	Not the tool for 2B LLMs (use LiteRT-LM)	Good: any <code>.tflite</code> converted from PyTorch, TensorFlow, or JAX. EmbeddingGemma supported directly	Yes: LiteRT Swift API and Core ML delegate; MediaPipe Text Classifier has an iOS guide	Active (LiteRT 2.x)
LiteRT-LM	Small LLMs (Gemma family) with tool calling	Mid-to-high tier with enough RAM. GPU and NPU backends	Not published	Gemma 4 E2B: ~1.1 GB mobile build (0.84 GB text-only); 2.9 GB Q4_0; 5.7 GB 8-bit (load memory). Gemma 3 1B: 658 MB int4 / 1,704 MB int8 files	Good: any <code>.litertlm</code> model (Hugging Face LiteRT Community); Gemma, FunctionGemma, and others	Swift API in early preview	Kotlin stable; recommended replacement for MediaPipe LLM Inference
MediaPipe LLM Inference API	Small LLMs (older API)	Stated as optimised for high-end (Pixel 8, Galaxy S23 or later)	Not published	As LiteRT-LM (<code>.task</code> format)	Being retired	Had iOS support; now moot	Maintenance-only ; migrate to LiteRT-LM

Runtime	Best at	Runs well on	Battery	~2B model size (disk / RAM)	Swapping models later	iOS reuse path	Status
ML Kit GenAI (Gemini Nano via AICore)	Zero-download on-device LLM on supported phones	Listed devices only, mostly flagships	Per-app battery quota enforced by AICore	Not in your APK; the system provides and updates the model	None: Google chooses the model and version	None (Android only). iOS has Apple's separate Foundation Models framework	Prompt API beta; Structured Output alpha; foreground only
ML Kit (classic) e.g. Entity Extraction	Dates, money, and card numbers in text	Broad	Not published	n/a	None (fixed Google models)	Yes: ML Kit has iOS SDKs	Entity Extraction in beta
ExecuTorch (PyTorch)	Any exported PyTorch model, including LLMs	Broad CPU (XNNPACK), plus Vulkan, Qualcomm, MediaTek, and Samsung Exynos backends	Not published	Depends on export and quantization (.pte)	Good if your models come from PyTorch or Hugging Face (Optimum ExecuTorch)	Yes: iOS guide with Core ML and XNNPACK backends	Core active; Android LLM Java/Kotlin API marked experimental
ONNX Runtime Mobile (+ onnxruntime-genai)	Classic ONNX models; LLMs via the GenAI extension	CPU/XNNPACK everywhere; NNAPI is device-dependent	Not published	Example int4 "cpu_and_mobile" builds exist (e.g. Phi-3-mini)	Good: many models convert to ONNX	Classic runtime: yes (Objective-C/C). GenAI: iOS "under development"	GenAI API in preview; Java binding needs a build from source
llama.cpp	LLMs in GGUF format	All tiers, CPU-first. The Android binding auto-detects CPU features for older, low-RAM devices	Not published	Gemma 4 E2B Q4_0: ~2.9 GB load memory (Gemma QAT GGUF)	Very good: the largest open-model format ecosystem	Yes (Metal on Apple)	Active; Android binding in examples/llama.android
MLC LLM	LLMs compiled per GPU	Needs a real mobile GPU; no emulator	Not published	Example config: Gemma 2B q4f16 ~3.0 GB estimated VRAM	Medium: each model must be compiled	Yes: iOS Swift SDK	Active; known UI-freeze issue on some Adreno GPUs with one weight layout

For weights alone: int8 takes about 1 byte per parameter and int4 about half a byte, before runtime overhead and the context (KV) cache. That explains most of the gaps in the size column.

Recommendation

- Classification and dedup tier (every device): LiteRT, through MediaPipe Tasks.** Use the Text Classifier for vertical and kind labels and the Text Embedder with EmbeddingGemma for similarity. It runs in the background on low-end phones, the model is just a file you can replace, and the same models can be reused on iOS.
- LLM tier (only where the phone supports it):** Gemini Nano through the ML Kit Prompt API when the device is on Google's list (no download; Google keeps it updated), and LiteRT-LM with a small Gemma model as the portable option on phones with enough RAM, preferably in batches while charging. Both sit behind one `OnDeviceLlmProvider` port.
- Future-proofing comes from the boundary, not the library.** Define `TextClassifier` (text → label and confidence) and `Embedder` (text → vector) ports in the Android domain layer, plus `LlmProvider` with `OnDeviceProvider` and `CloudProvider` adapters. Keep a small model registry (model id, version, runtime, file hash, download URL) so a model change is a config change. Swapping to ExecuTorch, ONNX Runtime, or llama.cpp later means writing one new adapter. Re-run the eval set (8.9) before any swap.

8.8 Recommendation: hybrid, on-device first

Tier	Runs where	Handles	Marginal cost to Pavan
0. Rules	Device	Allowlist check, known sender formats, OTP/personal drop, amount and date regex	Zero
1. Tiny models	Device (all phones)	Classifier and embeddings for classification and dedup	Zero (a one-time model download of a few hundred MB at most)
2. On-device LLM	Device (supported phones only)	Extraction when rules fail; ambiguous dedup tiebreaks	Zero, but limited by device capability, foreground-only (Gemini Nano), and quotas
3. Cloud LLM	Backend <code>ai-proxy</code> → hosted model (paid tier)	Hard cases only, with cloud-AI consent (Gate 3) and masked snippets	Pay per token; scales with use (see the 8.6 estimate)
4. Ask the user	Device	Anything still uncertain: "Is this a bill?"	Zero; the answer becomes feedback

- **Trust line:** "Everything is processed on your phone" is true by default. Cloud AI stays off by default, and only masked snippets leave the phone after the user opts in.
- **Background signals:** because Gemini Nano runs only in the foreground, signals that arrive in the background go through tiers 0–1 immediately. Items that need tier 2 are queued, marked "checking", and finished when the user opens the app, or by a LiteRT-LM WorkManager job with charging and idle constraints.
- **Cost:** on-device tiers cost nothing per message, and cloud cost scales with how often tier 3 is reached. The metric to watch is "% of signals reaching cloud", which the audit log records.
- **Swappability:** the same `LlmProvider` port shape is used on both sides. On Android there are `OnDeviceProvider` (ML Kit / LiteRT-LM) and `CloudProvider` (calls the backend). On the backend, there is one adapter per hosted model (section 13).

8.9 How to evaluate before choosing

1. **Build a labelled set** as described in section 5.3: at least 200 masked real samples for finance in the POC, with a duplicate-group id on each. Add samples for each new vertical before building its plug-in.
2. **Measure extraction:** per-field accuracy (kind, counterparty, date, amount) and the rate of schema-invalid outputs.
3. **Measure dedup:** precision and recall of "same item" decisions on the duplicate groups. Wrong merges hide real items, so weight precision higher for bills and bookings.
4. **Measure the tiers:** the share of signals each tier resolves, latency per signal on a low-end and a mid-range test phone, and cloud tokens per signal.
5. **Compare providers on the same set:** rules only, plus embeddings, plus Gemini Nano, plus LiteRT-LM Gemma, and plus each cloud model. Choose using the numbers, and re-run the set in CI on every prompt, model, or threshold change.

Only the anonymised set may go to a free-tier cloud API during experiments (8.6).

8.10 Learning path for Pavan (in order)

1. **What an LLM is:** [Google ML Crash Course – Introduction to LLMs](#).
2. **Embeddings and similarity:** [ML Crash Course – Embeddings](#), then [Gemini API – Embeddings](#).
3. **Structured output:** [Gemini API – Structured outputs](#) and [Kotlin serialization](#). Build: one cloud call that returns the finance plug-in's typed schema, using the free tier and synthetic data only.
4. **Tool calling:** [Gemini API – Function calling](#). Build: the dedup tiebreaker as one tool-style call.
5. **On-device AI on Android:** [Android AI overview](#), [Gemini Nano on Android](#), [ML Kit GenAI overview](#), [Prompt API get started](#), and [Structured output with the Prompt API](#).
6. **Open models on device:** [LiteRT-LM on Android](#), [EmbeddingGemma](#), [MediaPipe Text Embedder \(Android\)](#), [MediaPipe Text Classifier](#), and the [Google AI Edge Gallery app](#) to try models on a real phone.
7. **Backend side:** [Spring AI reference](#) (optional; a plain HTTP client behind `LlmProvider` is enough).
8. **Law:** [DPDP Rules, 2025 \(MeitY PDF\)](#) and the [PIB explainer](#).

8.11 Sources (checked 26 Sep 2026, IST)

- Google, *Overview of the ML Kit GenAI APIs* (device lists, per-app quota, foreground-only rule), developers.google.com/ml-kit/genai, page updated 17 Sep 2026.
- Google, *Get started with Prompt API* (beta status, under 4,000 input tokens, unlocked-bootloader limit), page updated 8 Sep 2026; *Generate structured output* (alpha), page updated 21 Jul 2026.
- Google AI Edge, *Get Started with LiteRT-LM on Android* (Kotlin API, GPU/NPU, tool use, initialisation time), page updated 4 Sep 2026; *LiteRT-LM NPU model table* (Gemma 3 1B sizes); *LiteRT-LM GitHub README* (language API status).
- Google AI Edge, *LLM Inference guide for Android* (maintenance-only notice, high-end device note), page updated 12 Jun 2026.
- Google AI Edge, *LiteRT overview* (LiteRT 2.x, iOS Swift and Core ML); *Text embedding guide for Android* (EmbeddingGemma support), page updated 17 Aug 2026; *Text classification task guide*.
- Google, *Gemma 4 model overview* (memory table), page updated 8 Jul 2026; *EmbeddingGemma model overview*, page updated 16 Apr 2026.
- Google, *ML Kit Entity Extraction* (entity types, languages, beta).
- Google, *Gemini Developer API pricing*, page updated 24 Sep 2026; *Gemini API Additional Terms of Service* (unpaid vs paid data use).
- Google Cloud, *Gemini 3.5 Flash-Lite model page* (US/EU ML processing regions).
- IDC via MediaBrief, *India smartphone market flat at 152 million units in 2025*, 17 Feb 2026 (price-band shares).
- MeitY, *Digital Personal Data Protection Rules, 2025*, G.S.R. 846(E), 13 Nov 2025 (Rules 1, 6, 15); PIB explainer, Nov 2025; Legal500 commentary on commencement dates (Sep 2026).
- PyTorch, *ExecuTorch docs* (Android backends, experimental LLM Java/Kotlin API, iOS backends).
- Microsoft, *ONNX Runtime mobile docs* and *onnxruntime-genai README* (preview API, platform matrix).
- ggml-org, *llama.cpp docs/android.md*; MLC, *MLC LLM Android SDK* and *iOS Swift SDK docs*.
- Apple, *Foundation Models developer documentation*.
- Reserve Bank of India, *Master Direction – Non-Banking Financial Company – Account Aggregator (Reserve Bank) Directions, 2016* (explicit, revocable consent); Department of Financial Services, *Account Aggregator Framework page* (FIP/FIU counts as on 31 Mar 2026).
- NPCI Bharat BillPay Ltd, *Intimation of Rebranding & Change in Logo of BBPS* (renamed Bharat Connect, announced 29 Aug 2024).

9. Security and compliance baseline (day one)

Data protection

- Data minimisation: store structured items, not raw messages. Raw text is processed in memory and discarded (section 10.1).
- Encryption at rest on device: Room with SQLCipher, key held in Android Keystore.
- Encryption at rest on server: managed Postgres encryption plus application-level encryption for sensitive columns (amounts, references) using a key from a secrets manager, never in code or the repo.
- Encryption in transit: TLS everywhere; certificate pinning once the domain is stable.
- Masking before any cloud AI call: account and card numbers, phone numbers, names, and addresses.

Access control

- JWT validation on every backend request; user id from the token, never from the request body.
- Postgres row-level security as a second wall, so one user's rows can't leak even if app code has a bug.
- Least-privilege DB roles: the app role can't bypass RLS; migrations run under a separate role.

Accountability

- Append-only audit log: what was read and when, allowlist changes, AI calls (tier, provider, purpose, masking applied), consent grants and revocations. Never message content.
- Consent records versioned against the exact text shown (section 11).

India DPDP Act alignment

- Clear notice and specific, informed consent per purpose; withdrawal as easy as granting. Each vertical and category is its own purpose (Gate 4).
- Purpose limitation: data from a source is used only for the verticals the user turned on.
- Right to access, correct, and erase: export and delete-my-data from Settings, which also deletes backend copies.
- Retention policy: raw data not retained; derived data deleted on request, on account deletion, or after a defined inactivity period (section 10.3).
- Breach readiness: a documented incident checklist, even if short, for the POC.
- Payments (P2): use an RBI-authorised payment aggregator; never store card data; follow their data-localisation requirements.

Engineering hygiene

- Secrets in GitHub Actions secrets and the cloud secret manager only.
- Dependency and secret scanning in CI (Dependabot/Renovate plus a secret scanner).
- Lint and static analysis (detekt, ktlint); unit tests on domain use cases, plug-ins, and masking; the eval set runs in CI.
- ProGuard/R8 on release builds; no sensitive data in logs or crash reports.

10. Security architecture – read path

This section covers the most sensitive moment in the app: when it reads a notification (the only source in the POC; SMS inbox and Gmail in later phases), and everything that happens to that data afterwards. It builds on the baseline in section 9, the consent ledger in section 11, and the AI tiers in section 8.

10.1 Every read is a trust boundary

Each read from an outside source crosses a trust boundary. Before a read happens, checks run in code, not just in the UI: the user holds a current consent event for that scope (section 11), the Android permission is still granted, the source is enabled in the privacy dashboard (section 12), and, for notifications, the posting app is on the user's allowlist. If any check fails, the read doesn't happen and the reason is written to the audit log.

Encryption in transit

- Later Gmail phase: API calls use Google's HTTPS endpoints; OAuth tokens are held encrypted on device (Android Keystore-backed) and are never sent to our backend.
- All app-to-backend traffic uses TLS with JWT authentication; certificate pinning is added once the domain is stable.
- SMS and notification content is read locally from the OS and never travels over a network in raw form.

No raw content persisted beyond what's needed

- Raw message text is processed in memory and discarded once classification and extraction finish.
- The only exception is an item that the rules and on-device models couldn't settle. Its text is kept, encrypted in Room, only until the user confirms it or the short retention window in 10.3 expires.
- Notifications from non-allowlisted apps, OTPs, personal conversations, and categories the user hasn't enabled are dropped at the filter and never written anywhere.

Explicit justification for each Android permission

Permission / access	Why the app needs it	When it is requested	If the user declines
None (demo mode)	Demo mode uses bundled sample data	Never	Not applicable
INTERNET	Sync structured items and the consent ledger with the backend	Install time (normal permission)	Not optional; nothing raw is sent over it
Notification access (<code>NotificationListenerService</code> , bound with <code>BIND_NOTIFICATION_LISTENER_SERVICE</code>)	Read notifications from apps the user has allowlisted (banks, billers, and later travel and shopping apps) as they arrive	After the notifications explainer and allowlist screen; the app deep-links to system settings, since this can't be granted by a dialog	App works with the other sources
READ_SMS (and RECEIVE_SMS)	Not requested in the POC. Later SMS-inbox phase: history and full text of bank SMS	Later phase only, after its own explainer, and only if Play's SMS policy allows it (section 5.1)	Notification access already covers most bank alerts
Gmail read-only scope (<code>gmail.readonly</code>)	Not requested in the POC. Later Gmail phase: bill, statement, booking, and delivery emails	Later phase only, after its own explainer and Google's verification (section 5.1)	App works with notifications and manual entry
POST_NOTIFICATIONS (Android 13+)	Show reminders and follow-up nudges	When the first reminder is created	Reminders appear only inside the app
Query all installed apps (<code>QUERY_ALL_PACKAGES</code>)	Not requested; the allowlist picker uses package names from seen notifications plus a declared list (section 5.2)	Never	Not applicable

Permission / access	Why the app needs it	When it is requested	If the user declines
Exact alarms (SCHEDULE_EXACT_ALARM)	Not requested in the POC	Never	Reminders use WorkManager and inexact alarms, which are accurate enough for due dates

Any new permission is a new consent version (section 11) and needs a row in this table before it ships.

10.2 Data minimisation: what leaves the device

The rule: **raw text stays on the phone. Only structured fields or masked snippets ever leave it.**

Data	Stays on device	Sent to our backend	Sent to a cloud AI provider
Notifications from non-allowlisted apps, personal chats, OTPs	Not even stored; dropped at the filter	Never	Never
Allowlisted notification text (and, in later phases, SMS and email bodies)	Yes, in memory only, then discarded (see 10.1)	Never	Never
Structured LifeItem (vertical, kind, counterparty, dates, amount, status)	Yes, encrypted in Room	Yes, if sync is on (sensitive columns encrypted at application level)	No
Masked snippet of a hard case (account and card numbers, phone numbers, names, and addresses removed)	Created on device	Passed through ai-proxy , not stored	Only if the user turned on cloud AI (Gate 3)
Embeddings used for dedup (section 8.4)	Yes	No	No
Gmail OAuth tokens (later phase)	Yes, Keystore-backed	Never	Never
Audit metadata (what was read, when, which tier handled it, whether cloud AI was used)	Yes	Yes	No

Masking runs on the device before anything is sent, and it has its own unit tests with real Indian formats (account number patterns, UPI IDs, card numbers, phone numbers, booking references). A snippet that still contains an unmasked pattern after masking is blocked, not sent.

10.3 Retention policy

These are proposed defaults for the POC. Confirm them against the DPDP Rules before any public launch (see the audit-log retention item in the Appendix).

Data	Where	Kept for	Purged when
Raw message text	Device memory	Seconds, for the length of processing	Immediately after extraction
Personal conversations and non-allowlisted notifications	Nowhere	Not kept	Dropped on arrival
Unresolved item awaiting user confirmation	Device (encrypted Room)	Up to 7 days	User confirms or rejects it, or 7 days pass
Open items (bills due, upcoming trips, deliveries)	Device and backend	Until done, dismissed, or deleted by the user	User deletes it, revokes the source and chooses to delete derived data, or deletes the account
History (paid bills, completed items)	Device and backend	While the account is active	User deletes items or the account, or after a defined inactivity period (proposed: 12 months, with a warning first)
Sale events and their embeddings	Device	Until the event ends, plus a short grace period (proposed: 7 days)	Automatically after that, or when the category is turned off

Data	Where	Kept for	Purged when
Masked snippets sent to cloud AI	Our backend: not stored. Provider: governed by its terms	Our side: not retained	Use only a paid provider tier whose terms exclude training on user content; re-check terms before each provider change
Audit log (metadata only, no message content)	Device and backend	At least as long as the DPDP Rules require for processing logs	After the required period; not erased by delete-my-data while legally required, which the user is told
Consent ledger	Device and backend	Account lifetime, plus as long as needed to prove lawful processing (section 11)	After that period
Backups (managed Postgres)	Provider	The provider's backup cycle	Deleted rows age out as backups rotate; the privacy notice says so
Crash and performance logs	Crashlytics / backend logs	Provider default	Never contain message content, amounts, or counterparties

Purging is a scheduled job on both sides (a WorkManager job on the device and a scheduled backend task), and each purge run writes an audit entry with how many records it removed.

10.4 Staying within Android's background and notification-access rules

- **Notification access is user-granted and system-bound.** The system binds `NotificationListenerService` only after the user enables it in settings, and the user can turn it off at any time. The app checks this state on every launch, reflects it in the privacy dashboard, and never pressures the user with repeated prompts.
- **Allowlist before content.** The first thing the listener does is check the posting app against the allowlist; anything else is ignored without reading the text (section 5.2).
- **Protected notifications are respected.** Recent Android versions hide sensitive content, such as one-time passwords, from notification listeners. The app doesn't try to get around that, and it drops OTPs anyway.
- **No long-running background services.** The system manages the listener, so the app needs no foreground service of its own. Everything else (re-scans, follow-ups, purges, the heavier LLM tier) runs as WorkManager jobs, which follow Doze and battery-saver rules. Heavy jobs run only while charging and idle (section 8.8).
- **Foreground-only AI stays foreground-only.** Gemini Nano refuses background use, so background items go through rules and tiny models, and anything harder waits until the app is opened (section 8.8).
- **Listener reliability.** Some OEM battery managers can stop the listener. The app detects disconnects, requests a rebind, and shows a banner with guidance instead of silently missing items (section 5.4).
- **SMS policy (later phase).** The POC doesn't request `READ_SMS`. An SMS-inbox phase ships only if the Play permission declaration is approved or it's distributed outside Play (section 5.1).
- **Play Store user-data rules (public release).** A prominent in-app disclosure appears before each sensitive permission request (the explainer screens in section 12 serve this purpose). The Play Data safety form matches the tables in 10.2 and 10.3, and the privacy policy says the same thing in plain language.
- **Reminders without exact alarms.** Reminders don't need minute-level precision, so the app uses WorkManager and inexact alarms and avoids the restricted exact-alarm permissions.

10.5 User confidence: how we communicate this

Every message below is backed by a code path and a ledger or audit entry, so the app never claims more than it does.

- **What is read:** each source's explainer (section 12) says what is read and why, before the permission prompt. The privacy dashboard repeats it next to each grant, along with the list of allowlisted apps.
- **What leaves the device:** the headline line is "Your messages and notifications are processed on your phone. Only masked snippets leave it, and only if you turn cloud AI on." The dashboard shows a plain count, such as "0 snippets sent to cloud AI this week", taken from the audit log.
- **What is stored:** a "What we keep" screen lists the retention table from 10.3 in plain language: structured reminders and history, yes; raw messages and personal conversations, never; sale events, deleted a week after they end.
- **Control:** one-tap revoke per source and per app, data export, and delete-my-data are always one screen away. Revoking writes a ledger event and stops reads immediately.

- **Changes:** when a consent version changes what is read, stored, or sent, affected users see a short "what changed and why" note before anything new happens (section 11).
- **Proof over promises:** a public-facing security page (after the POC) summarises 10.1–10.4, and the portfolio README links to this section so reviewers can check the design.

11. Consent versioning

Consent is a versioned, append-only ledger from day one, even though v1 starts with only a few screens. Retrofitting versioning later is painful; recording it now costs one table and one event per grant.

When the ledger starts. Demo mode (Gate 0) processes no personal data, so it writes nothing to the ledger. The first event is the account grant (Gate 1). After that, every source, allowlist change, category or vertical, and cloud-AI decision is its own event, recorded as the user makes it, one source at a time (section 3).

Consent events

- Every grant, update, and withdrawal is stored as an immutable event. Nothing is overwritten; the current state is derived from the latest event per user and scope.
- Each event stores:
 - `consent_version` (for example `1.0`, `1.2`, `2.0`)
 - `timestamp` (UTC, server-assigned) and the device's local time
 - `what_was_shown`: the exact explainer or notice text, or a content hash plus a pointer to the stored copy of that version's text and screen
 - `scopes` granted or declined: sources (notifications in the POC; SMS inbox and Gmail later), categories and verticals (finance, sales and events, and later travel, deliveries, appointments, renewals), cloud AI, and later documents and payments
 - `scope_detail` where needed, for example the app package added to or removed from the notification allowlist
 - `action` (granted, declined, withdrawn, re-confirmed)
 - `user_id`, app version, platform (Android now, iOS later), and locale
- Consent texts are stored as versioned records, so any past event can be shown alongside the exact wording the user saw.

Server-side audit trail (DPDP)

- The server holds the authoritative ledger; the device keeps a local copy for offline gating and syncs every event to the backend.
- The ledger is append-only (no updates or deletes from the app role) and feeds the audit log, so the service can show when, how, and on what terms consent was given or withdrawn.
- On account deletion, personal data is erased, while a minimal consent record is retained only as long as needed to prove lawful processing, as described in the retention policy.

Re-prompting on new versions

- Each version declares which scopes it changes. Version bumps follow a simple rule:
- Minor (`1.1`, `1.2`, `1.3`): wording or a change to one scope. Re-prompt only users who hold consent for an affected scope.
- Major (`2.0`): a new purpose, a new data source, or a new processor. Re-prompt all users whose consent covers anything that changed.
- **New verticals are offered, not imposed.** A new vertical is a new purpose, so it arrives as a new opt-in scope. Users who don't opt in are never interrupted and their existing scopes keep working.
- Until an affected user accepts the new version, processing for the changed scope pauses; unaffected scopes keep working.

iOS later

- iOS uses the same ledger, the same event schema, and the same versioning rules.
- Only the prompts and available scopes differ, because Apple is stricter about background access and message reading. SMS and notification reading as done on Android isn't available, so iOS consent screens will offer a smaller set of scopes (such as Gmail and manual entry), each recorded with `platform = ios`.

12. Trust and transparency

The app asks for some of the most sensitive data on a phone, so trust is designed as a feature, not a disclaimer. Each element below ties back to the consent ledger in section 11, so what the user sees and what the system records always match.

Demo first (earn trust before asking)

- The first run shows the full journey on labelled sample data, with no account and no permissions (section 3). The user decides to connect real data only after seeing the value.
- Sample and real data never mix, and demo cards for verticals the app doesn't yet process for real are labelled "Coming soon on your data".
- *Ledger link*: nothing is recorded during the demo because nothing is read. The ledger starts at the first real grant.

Lead with on-device processing

- The first line of the "Try it on your phone" step and the store listing: "Your messages and notifications are processed on your phone. Only masked snippets ever leave it, and only if you turn cloud AI on."
- This is backed by the architecture in sections 6.3 and 8.8: filtering, classification, extraction, and dedup run on device; cloud AI is off by default and receives masked text only.
- *Ledger link*: the cloud-AI scope is its own consent event. Any change to what leaves the device (a new field, a new AI provider) is a new consent version and re-prompts only users who have cloud AI on.

Show the work before asking, one source at a time

- Before each permission prompt, a short screen for that source explains three things in plain language:
- **What is read** (for example, "notifications from the apps you pick below, to find bills, due dates, and amounts").
- **What is never stored** (personal conversations, OTPs, notifications from apps you didn't pick, and the raw text of any message).
- **How to revoke** (one tap in the privacy dashboard, which also offers to delete what was derived from that source).
- After the first grant, show a real result from the user's own device (a detected bill, with account numbers already masked) before offering the next source.
- *Ledger link*: the exact explainer text shown is what gets stored as `what_was_shown` in the consent event, so the record proves the user saw this explanation.

In-app privacy dashboard

- One screen listing every grant: each source, the notification allowlist (with per-app remove), each category or vertical, cloud AI, and later documents and payments.
- For each one: status, the date it was granted, the consent version it was granted under, and a one-tap **Revoke** button.
- Recent activity from the audit log in plain language ("Checked 12 new notifications from 3 apps today; 2 bills found; nothing sent to cloud AI"), plus data export and delete-my-data.
- *Ledger link*: the dashboard reads its state from the ledger, not from a separate settings table. Revoking writes a `withdrawn` event, processing for that scope stops immediately, and the event syncs to the server audit trail.

When things change

- A new consent version is explained with a short "what changed and why" note, shown only to affected users (per section 11's re-prompt rules).
- Declining a new version pauses only the affected scope; the rest of the app keeps working.
- *Ledger link*: the "what changed" note is stored with the new version's text, so each user's history shows every version they accepted or declined.

POC scope

- Demo mode ships on day 2. Explainers, the allowlist screen, and the on-device message line ship on day 4, and the consent ledger on day 5.
- A basic privacy dashboard (grants, allowlist, version, revoke) ships with export and delete-my-data on day 13 and is polished on day 19; the plain-language activity feed can follow after the POC.

13. Free-to-paid upgrade path per service

Service	POC (free)	When to upgrade	Paid path	Code change needed
Postgres	Supabase or Neon free tier (small storage, may pause or scale to zero when idle)	Real users, need for always-on DB, backups, more storage	Same provider's paid plan, or AWS RDS / Aurora	Connection string only (Terraform variable)
Backend hosting	Cloud Run free tier or AWS free credits, scales to zero	Cold starts hurt UX, steady traffic	Min instances > 0, or ECS/EKS on AWS	None; same container image
Auth	Firebase Auth / Auth0 free tier	User count or enterprise features (SSO, MFA policies)	Paid tier of the same provider	None if JWT validation is behind <code>AuthProvider</code>
On-device AI	LiteRT / MediaPipe models, Gemini Nano, LiteRT-LM	Not applicable (no per-use cost)	Better or larger models via the model registry	Model registry entry; new adapter only for a new runtime
Cloud AI (opt-in)	Free tier with synthetic data only during development; paid tier for any real user data, with a small daily cap	Accuracy needs or volume grow	Stronger model, batching, or a self-hosted model	New adapter implementing <code>LlmProvider</code>
Object storage (P1)	Supabase Storage or Firebase Storage free tier	Vault usage grows	S3 with KMS encryption	New <code>StorageProvider</code> adapter
Push notifications	Not needed in the POC (local reminders); Firebase Cloud Messaging (free) later	Rarely needs upgrading	Same	None
CI/CD	GitHub Actions free minutes	Private repo minutes run out	Paid minutes or self-hosted runner	None
Monitoring	Spring Actuator + provider logs + Firebase Crashlytics (free)	Need tracing and alerts	Grafana Cloud, Datadog, or CloudWatch	Add exporter config
Infra	Terraform with local or free remote state	Team collaboration	Terraform Cloud or S3 state backend	Backend block change
Payments (P2)	Sandbox of an Indian payment aggregator	Go-live	Live keys, per-transaction fees	Config and compliance, not code

Guiding rule: every vendor sits behind an interface and every environment difference is a Terraform variable or a config value, so moving from free to paid is a config change, not a rewrite.

14. Monetization (next steps, not POC scope)

A conceptual sketch to guide product decisions, not a final business plan. Prices are suggestions to test with real users, not market data.

14.1 Tiers

	Free	Pro (monthly subscription)
Verticals	Finance (the first vertical) and sales and events; as later verticals launch, one of the user's choice	Every vertical as it launches: finance, travel, deliveries, appointments, renewals, sales and events
Classification	Rules and on-device models	The same, plus opt-in cloud AI for hard cases
Life feed (Today / Upcoming / Overdue) and manual items	Yes	Yes
Bill tracking and auto-match "paid"	Limited (for example a small number of tracked bills)	Unlimited
Reminders and follow-ups	Basic reminders	Smart follow-ups, snooze, re-check of new messages, travel-day and delivery-day alerts
Dedup across sources and teasers	Yes	Yes
Document vault with due-date and clause extraction	No	Yes
Agent features (persona, patterns, personalised suggestions)	No	Full
History	Short window (for example 3 months)	Long window (for example 2 years or more)
Demo mode, export, and delete-my-data	Yes	Yes

Privacy features (consent controls, the allowlist, the privacy dashboard, on-device processing, export, deletion) are the same on both tiers. Privacy is never a paid upgrade, because that would undercut the trust positioning in sections 10 and 12.

14.2 Suggested Pro price: ₹99–₹199 per month

- **Why this range:** it sits where many Indian consumer subscriptions are priced, so it reads as an everyday app subscription rather than a financial product. One avoided late fee or missed premium can cover months of the subscription, which is the core value message. The life-wide verticals strengthen it by adding everyday value (trips, deliveries, renewals) beyond money.
- **Starting point to test:** launch near ₹99/month with an annual plan at a discount (roughly two months free) to improve retention, then test a higher price once the vault, more verticals, and agent features mature.
- **Payments:** recurring UPI AutoPay (e-mandate) and cards through an RBI-authorised payment aggregator, plus Google Play Billing where Play policy requires it for digital subscriptions sold in the app. Check Play's current billing rules for India before launch.
- **Cost check:** per-user AI cost is low because most processing is on-device, and the cloud fallback estimate in section 8.6 is a small fraction of a rupee-level subscription. Pro margins depend mainly on payment fees, storage for the vault, and support.

14.3 Privacy promise (non-negotiable)

Revenue comes from Pro subscriptions. The app does not sell user data, insights about users, or access to users, and no revenue plan depends on it. User content (message text, email bodies, notifications, documents, names, account numbers, individual records) never leaves the user's control for any commercial purpose. This holds on both tiers and is not traded for growth. Privacy is never a paid upgrade.

15. Portfolio angle and resume line

What a reviewer for a senior role should be able to see in the repo

- **A working demo:** a 2-minute video and a release APK showing demo mode, then the real finance flow end to end from notifications (bill notification, merged with the biller app's notification, reminder, payment notification, marked paid).
- **A feasible data pipeline with measured accuracy:** the labelled-set method, the eval script in CI, and the committed accuracy report (per-field accuracy, dedup precision/recall, share of signals per AI tier).
- **A defensible privacy story:** on-device-first processing, the per-app allowlist, one-source-at-a-time consent with a versioned ledger, and the read-path tables in section 10.
- **Decision records (ADRs):** auth provider, notifications-only POC, thin backend, fixed pipeline over an autonomous agent, on-device runtime and model registry, allowlist and SMS-app rule, and demo-first onboarding.
- **Tests:** unit tests for plug-ins and masking, repository tests with encrypted Room, RLS policy tests on the backend, UI tests driven by the demo fixtures, and the eval set as a regression suite.

Resume one-liner

Built Orbit One, a consent-first, Android-first personal assistant in Kotlin (Jetpack Compose, on-device AI with LiteRT and Gemini Nano, a thin Spring Boot modular monolith, Postgres with row-level security, Terraform, GitHub Actions). It turns app notifications into one deduplicated list of what's coming up, starting with bills, SIPs, and insurance, with a pluggable vertical design ready for SMS and email sources. It processes and discards notification content on the device behind a per-app allowlist and a versioned consent ledger, and its extraction accuracy is measured on a labelled set of real Indian notifications. It is designed for India's DPDP Act and to move from free tiers to paid infrastructure with config-only changes.

16. On-device analytics: is it working for real users?

The app keeps a small event log on the phone that counts four things. The first is **catches**, actionable items extracted and not corrected. The second is **misses**, notifications that should have become items but didn't; these are counted when the user quick-adds something the listener had seen, or marks a skipped or "promotion" notification as real. The third is **corrections**, the "That's not right" taps from section 8.5. The fourth is **dedup merges**, along with merges the user undid. Only counts and structured labels are kept, never notification text, and nothing is uploaded. The privacy dashboard shows the counts to the user, and a tester can choose to export them to Pavan. This shows whether the POC works on real phones and real notifications, not just on the labelled test set.

17. Performance requirements (first-class)

Performance is a requirement with budgets, not a polish item. These are proposed targets for the POC, to be confirmed or tightened once real-device numbers exist (ADR-004 in the repo).

Requirement	Budget	How it's measured
No perceptible jank	The UI renders at 60 fps (16 ms per frame): under 1% slow frames and no frozen frames (over 700 ms) while scrolling the feed or opening the detail view and bottom sheet. The main thread never does disk, database, network, or model work	JankStats in debug builds; a Macrobenchmark for feed scroll and item open on a mid-range phone; StrictMode (disk and network on the main thread) set to crash in debug builds
Ingestion-to-display latency	Under 2 seconds at p95 from the notification arriving at the listener to the card being visible, with the app in the foreground, on the rules and tiny-model path. On-device pipeline processing alone should take under 200 ms at p95 per notification. With the app in the background, the item is saved within 2 seconds and is on screen at the next open. Items that wait for the on-device LLM or cloud fallback appear first with a low-confidence flag and are updated in place	Timestamps at each pipeline stage in the ingest log (timings only, no text); an instrumented test that posts a notification from the companion app and waits for the card
Throughput	At least 500 notifications a day per user with no growth in latency or memory, including bursts of 50 notifications within 10 seconds (for example, a sale day or a grouped bank batch) with none dropped or processed twice	Burst and soak replays of the synthetic set through the pipeline; the listener hands work to a bounded queue off the main thread

Design rules that follow from this. The listener callback only copies the notification into a bounded in-memory queue and returns. All parsing runs on background dispatchers. Room writes are batched. The feed reads through a paged, observable query so that a new item updates one card rather than redrawing the list. Model loading is lazy and never happens on the main thread.

Eval report link. The synthetic generator's eval report adds a performance section that measures against these budgets. It reports per-stage and end-to-end pipeline latency (p50, p95, p99), a replay of 500 notifications as a day's load, a 50-in-10-seconds burst test, and a pass or fail for each budget. Numbers from the JVM eval are indicative; the on-device Macrobenchmark and instrumented latency test are the numbers that count.

18. Future feed improvements (post-POC, not in week 1 scope)

These ideas came up on 26 Sep 2026 and are recorded here so they aren't lost. None of them are in the POC plan, except where section 19 notes otherwise.

1. **Smart ranking.** The app learns which categories the user acts on most (marks paid, opens, sets reminders) and shows those first. Learning uses only the on-device counts from section 16, so nothing leaves the phone.
2. **"Needs attention" section.** A strip at the top of the feed pulls out time-sensitive items, such as a bill due today, an overdue payment, or a delivery arriving now. If the two-layer home in section 19 is adopted, a simple version of this ships in the POC as the Today's Priorities card.
3. **Quiet hours.** Low-priority alerts from Orbit One itself (sale items, informational updates) are held overnight and delivered as one morning summary. Orbit One can only batch its own alerts, not other apps' notifications. Money and time-critical reminders, such as a bill due that day, always go through.
4. **Category filter chips.** Chips on the home feed (Bills, Payments, Reminders, Deliveries, Sales) let the user narrow the list. If section 19 is adopted, these chips ship in the POC on the full feed.
5. **Urgency sorting within a time window.** Within Today, Upcoming and the other windows, items are sorted overdue first, then due soon, then informational items, instead of purely by date.
6. **Weekly digest.** Once a week, a summary shows what the user handled (bills paid, deliveries received) and what slipped (items that went overdue or were missed). It is built on the device from the `LifeItem` history and the section 16 counts, and nothing is sent anywhere.
7. **Quick actions from a card or notification.** Actions like Mark paid, Snooze, and Remind me tomorrow can be done directly from a feed card or from Orbit One's own reminder notification, with voice or quick reply where Android supports it. These actions only update Orbit One's own records. They never pay a bill or reply to anyone on the user's behalf.
8. **Calendar sync.** Due dates can be shown in the user's calendar so everything is in one place. This is a separate opt-in with its own consent scope recorded in the ledger (section 11). By default it writes only a title and a date, with no amounts or account details, and it can be turned off at any time.

These all build on data the POC already produces: the `LifeItem` due date, status and category, plus the on-device analytics. Each should be checked against the performance budgets in section 17, especially feed rendering with ranking applied.

19. Home screen structure: two layers (proposed for the POC)

This is the proposed home structure for the POC, suggested on 26 Sep 2026. The home screen has two layers. The first is a short summary the user can take in at a glance, and the second is the full list of items underneath it.

Layer 1: Glance (the home screen). This is what the user sees when they open the app.

- A greeting at the top, such as "Good Morning, [Name]", with a one-line subtitle ("Here are your priorities for today").
- A **Today's Priorities** card that groups urgent items by category, such as bills due and insurance premiums. Each row shows the item, where it came from (the bank, biller or insurer), the amount, and its due state (due today, overdue, or due tomorrow), so it's clear why it's a priority. A count badge on the card matches the number of rows.
- An **Upcoming Reminders** card for items coming up in the next few days, such as a SIP instalment in two days or a delivery arriving.
- A **See all** link on each card.

Layer 2: Full feed. This is the existing card-list feed from the POC, with category filter chips on top (All, Bills, Payments, Reminders, Deliveries). The user reaches it through **See all** or by tapping into a category. The item detail view and the "That's not right" feedback sheet (section 8.5) live here, unchanged.

The bridge between the layers. **See all** on a priority card opens the full feed already filtered to that card's category. For example, See all next to the bills opens the feed with the Bills chip selected, and the back arrow returns to the Glance screen.

Reference design. The home screen screenshot shared on 26 Sep 2026, saved as `orbit-one-home-reference.png` next to this plan, is the visual reference for Layer 1. The Orbit One mockups keep its layout but use the Orbit One colours and category icons, and add the due state to each priority row. The updated mockups are `01-home-glance` (Layer 1) and `02-full-feed` (Layer 2), each in light and dark, in the Orbit One mockups folder.

How it fits the rest of the plan. Both layers read from the same `LifeItem` data (due date, status, category, amount), so no new data or permissions are needed. The Glance cards show a small, fixed number of rows each and are built from the same paged Room query as the feed, so they stay within the section 17 budgets. Adopting this structure brings two ideas from section 18 into the POC in a simple form: the "Needs attention" strip (item 2) becomes the Today's Priorities card, and the category filter chips (item 4) become part of the full feed. Smart ranking and the other section 18 items stay post-POC.

20. Demo flow and future feature introductions

This section was added on 26 Sep 2026. The demo flow in 20.1 uses only features the POC builds, so it can be used for the week 4 demo (section 4, day 20). The AI assistant in 20.2, the rollout strategy in 20.3, and the vault principle in 20.4 are **future work (post-POC)**. They record the vision so that POC design choices don't block them, and none of it is in the four-week plan.

20.1 Demo flow: showing the POC end to end

A live demo follows one bill from the moment its notification arrives to the moment the user's correction is recorded. It takes about three minutes.

1. **A notification arrives.** The presenter sends a sample bill notification from the companion test app (for example, "HDFC credit card bill of ₹12,450 due on 28 Sep"). The companion app is on the allowlist, so no real account or personal data appears on screen. The presenter points out that the notification is read and processed on the phone.
2. **The Glance screen shows it.** Within the 2-second budget in section 17, the bill appears in the Today's Priorities card (section 19) and the badge count goes up. Optionally, the presenter sends a second reminder for the same bill to show the two being merged into one item instead of appearing twice (section 8.4).
3. **The user opens the detail.** Tapping the row opens the item detail, which shows what was extracted (amount, due date, biller, category), the source app, when it arrived, and "why this is here".
4. **"That's not right".** The presenter opens the feedback sheet and makes one correction, such as fixing the category or the amount. The card updates immediately.
5. **The feedback loop closes.** The privacy dashboard shows the correction counted on the device (section 16), and the presenter explains that corrections are saved as structured fields only and tune that user's rules (section 8.5). Marking the bill as paid then clears its follow-up reminder.

Backup plan. If the live notification path fails during a demo (for example, the listener has been stopped by the phone's battery manager), the presenter switches to demo mode with sample data (section 3), which shows the same journey with no permissions and no network. Running the demo once in airplane mode also shows that the core flow works without the cloud.

20.2 AI assistant (post-POC)

The vision. A conversational assistant inside Orbit One that the user can ask questions in plain language, such as "When is my next bill due?", "What did I spend this month?", or "When does my car insurance expire?". It covers bills, payments, reminders, and documents in the vault.

How it works.

- It answers only from the data already on the phone: the `LifeItem` records the pipeline extracts, payment matches, reminders, and vault document details (type, tags, expiry and renewal dates).
- The on-device model turns the question into a lookup on the local database, then writes a short answer in plain language. Each answer lists the items it used, and the user can tap one to open it.
- **It never sends data off the device.** It uses the on-device model only (Gemini Nano or a small model on LiteRT-LM, section 8.7) and never the cloud fallback, even if the user has turned on cloud AI for extraction. On phones without a capable on-device model, it offers a short list of fixed questions answered straight from the database, or it isn't offered at all.
- It only reads. It can open an item or set a reminder when the user asks, but it never pays, sends messages, or changes data without the user confirming.

Caveats to state honestly.

- Answers are only as complete as what Orbit One has caught. "What did I spend this month?" covers only the payments seen from allowlisted apps, and the answer says so ("from the payments Orbit One saw").
- When the data doesn't contain the answer, it says it doesn't know instead of guessing.
- Accuracy needs its own labelled question set and eval, like the extraction eval, before it ships.

20.3 Introducing new features without clutter (post-POC)

New features such as the full vault (with clause extraction, P1), the AI assistant, and later a payment gateway are added without changing the core experience. The Glance screen and the feed stay as they are.

- **Progressive disclosure.** A new feature is suggested only when it's relevant to that user. For example, once the user has several insurance items, a single dismissible card suggests keeping the policy documents in the vault. Suggestions never appear inside the Today's Priorities card, there is at most one at a time, and a dismissed suggestion doesn't come back.
- **Opt-in phases.** Each feature rolls out in stages: Pavan and internal testers, then an opt-in beta, then everyone. Each stage sits behind a remote feature flag, and it moves forward only when its accuracy and reliability targets are met (in the same way as the Phase 2 triggers in the Appendix).
- **Settings-gated.** Every optional feature has its own toggle in Settings and is off by default. Turning one off hides it and offers to delete the data it created.
- **Consent per feature.** Any feature that reads new data or acts for the user gets its own scope in the consent ledger (section 11), such as documents, the assistant, or payments.
- **Order.** The vault comes first (basics are already in the POC, week 3), then the AI assistant, then payments.
- **The payment gateway comes last and needs a regulated partner.** Paying bills from inside the app needs an RBI-regulated payment partner or bank and Bharat Connect integration (section 5.3), a legal review, and explicit user confirmation for every payment. Orbit One never pays automatically. Until that exists, actions like Mark paid only update Orbit One's own records (section 18, item 7).

20.4 Vault introduction: documents tied to your life (post-POC design principle)

The vault should feel like a natural part of Orbit One, not a separate feature added on top. It grows out of the same story as the rest of the app: things happening in the user's life, caught from their notifications. The framing is "**your documents, tied to your life**", not "a document locker".

- **Documents are linked to items.** Insurance policies, warranties, and receipts are linked to the bills, payments, and reminders the pipeline already extracts. A bill's detail view can show its linked document (for example, the LIC premium shows the policy). A warranty reminder points to the warranty card or purchase receipt in the vault, and a renewal reminder points to the current policy.
- **Entry points are contextual.** Documents are added and opened from a card, an item's detail view, or a reminder ("Attach a document", "View policy"). There is no separate vault tab that feels disconnected from the feed. A plain list of all documents is still available from Profile for users who want to browse, but it is not the main way in.
- **Suggestions follow the same rules as 20.3.** When a new item looks like it has a document worth keeping (a new insurance policy, a large purchase with a warranty), Orbit One may suggest attaching it once, and the user can dismiss it.
- **Links are made by the user or suggested, never assumed.** Orbit One can suggest a link when the biller, policy number, or product matches, but the user confirms it. Documents stay encrypted on the phone, as in the POC vault basics (section 4, day 15).

What this means for the POC. The POC's vault basics (week 3) should store an optional link from a document to a `LifeItem` from the start. That makes the linked-document view and contextual entry points cheap to add later, without changing the data model.

21. Motion and transition principles (design principle for the POC and beyond)

This section was added on 26 Sep 2026, taking inspiration from the Arc and Dia browsers. It applies to the POC and to every later feature. The aim is an app that feels intuitive: the user always knows where they came from and where they are going. The motion tokens that go with it are in the design token sheet (`tokens.md` , "Motion").

1. **Motion is navigation, not decoration.** Every transition answers the question the previous screen left open. Tapping See all on "Today's priorities" answers "what else is due?", so the motion should show the card becoming that list. An animation that doesn't help the user follow where they went is left out.
2. **Glance to feed unfolds instead of jumping.** When the user taps See all or a category, the Glance card expands into the full feed. Its header becomes the feed's title, its rows stay in place and become the first feed cards, and the remaining items and the filter chips fade in around them. Going back folds the feed back into the card.
3. **Detail slides up as part of the same surface.** Tapping an item grows its row or card upward into the detail view, so it reads as the same item opening up rather than a new screen. The "That's not right" sheet rises from the bottom of that same view. Back, including Android's predictive back gesture, plays the same motion in reverse.
4. **Fast and purposeful.** Every transition finishes in under 300 ms: about 200 ms for small changes (a chip filter, a toggle, a row collapsing after Mark paid) and up to 300 ms for the larger container changes above. There are no looping, bouncing, or decorative animations.

How it's built. In Jetpack Compose this maps to shared element transitions (`SharedTransitionLayout` with `sharedBounds` for the card-to-feed and row-to-detail changes), `AnimatedContent` and `Modifier.animateItem()` for list and filter changes, and the Material 3 bottom sheet for feedback. Predictive back is supported from the start.

Guardrails.

- **Performance comes first.** Animations must hold the section 17 budget of 60 fps with under 1% slow frames. The feed-scroll and item-open Macrobenchmarks include the transitions. No data is loaded while a transition runs; the next screen's data is already in memory from the same Room query.
- **Reduced motion is respected.** When the user has turned off or reduced animations in Android settings, transitions become an instant change or a short fade.
- **Mockups are static.** The v0.2 mockups show the start and end states only. The motion itself is specified here and will be checked on a real device.

Appendix: open decisions

Decided (26 Sep 2026)

- **POC length: four weeks full-time is the committed base.** One or two extra weeks are allowed only for Phase 2 work. The three-week and two-week cuts in section 4 remain fallbacks only.
- **Phase 2 order: Gmail (read-only) first, then SMS inbox.** Gmail is expected to be the easier permission for users to turn on.
- **Phase 2 is the last thing built and doesn't block the POC.** Gmail and SMS inbox access improve how many items the app catches, but the POC ships without them.

Still open

- Firebase Auth vs Auth0 (week 1 ADR).
- Supabase vs Neon (Supabase bundles storage and auth options; Neon is pure Postgres with branching).
- Exposed vs jOOQ (Exposed is Kotlin-native; jOOQ is SQL-first and strong with Postgres features).
- Which hosted LLM for the POC (choose on price per call and quality on the labelled set of real Indian formats; paid tier only for real user data).
- On-device inference runtime. Recommendation: LiteRT (through MediaPipe Tasks Text Classifier and Text Embedder with EmbeddingGemma) for the classification and dedup tier on every phone. For the LLM tier, Gemini Nano via the ML Kit Prompt API where the device supports it, and LiteRT-LM with a small Gemma model elsewhere. Reasoning: a tiny classifier or embedding model runs in the background on low-end phones, while Gemini Nano is limited to listed, mostly flagship devices and to foreground use. The runtime sits behind `TextClassifier`, `Embedder`, and `LlmProvider` ports with a model registry, so ExecuTorch, ONNX Runtime, or llama.cpp can be swapped in later with one adapter. Details and comparison in section 8.7.
- Which vertical comes second after finance: travel bookings (clear dates, high cost of missing, common confirmation formats) or deliveries (very frequent, low stakes). Suggest travel, and decide from tester data.
- Per-app allowlist defaults: suggested but not pre-selected (recommended), or pre-selected finance apps with a review screen. Also decide which app categories are never suggested (messaging and social apps).
- Accuracy targets per vertical (per-field accuracy and dedup precision) that must be met before a vertical ships.
- Promotional filter targets: the maximum acceptable rate of real bills or payments wrongly skipped as promotions (should be close to zero), and whether promotions are dropped by default or shown in a muted "skipped" list (section 8.2).
- Phase 2 trigger thresholds: the accuracy targets, user-retention and trust signals, and completed platform reviews required before offering the Gmail opt-in publicly, and then the SMS inbox opt-in (section 2, "Access progression"). The order itself is decided (see above).
- POC distribution: Play internal testing or direct APK. With notifications only, no SMS permission is needed, so internal testing on Play is an option.
- Backend sync in the POC: sync structured items (backup and history) or keep items on device only, with just the ledger and audit on the server. Later option: encrypt synced items with a device-held key so the server stores only ciphertext.
- Cloud LLM provider tier and region: paid tier only for real user data (free-tier terms forbid personal information); decide whether US/EU processing is acceptable or an India-region option is needed (section 8.6).
- On-device LLM model choice (for example Gemma 4 E2B vs Gemma 3 1B on LiteRT-LM) and whether to download it only on Wi-Fi and while charging.
- Minimum supported device (Android version, RAM) for the full experience vs a rules-and-embeddings-only mode on low-end phones.
- Sales and events category: ship in the POC as a stretch or after; which shopping apps are suggested (none pre-selected); the consent text for the new scope.
- Dedup similarity thresholds and date-window tolerance: set from the labelled eval set, possibly per vertical (stricter for bills and bookings).
- Handling background signals that need an LLM: queue until the app is opened (Gemini Nano is foreground-only) or a charging-and-idle LiteRT-LM WorkManager job.
- Two-layer home screen (section 19): confirm it as the POC home structure, and set the rule for what counts as a priority (for example, overdue or due within one day) and how many days ahead Upcoming Reminders looks.
- AI assistant (section 20.2, post-POC): the minimum on-device model it needs, and whether phones without one get fixed questions or no assistant at all.
- Payment gateway (section 20.3, post-POC): which regulated partner, and what legal review is needed before in-app payments.
- Performance budgets (section 17): confirm or tighten the proposed targets (60 fps with under 1% slow frames, under 2 s p95 notification to card, 500+ notifications a day with 50-in-10-second bursts) after the first mid-range-phone measurements.

- Audit-log retention vs delete-my-data: check DPDP Rules, Rule 8(3) (minimum one-year retention of processing logs, in force from the 18-month phase) against the retention policy in section 10.3.
- Regulated finance data routes (P2): whether and when to pursue Account Aggregator access through a regulated partner, and Bharat Connect bill fetch through a participating bank or operating unit (section 5.3).
- Order and timing of later sources: SMS inbox (history, full text; needs Play's SMS permission declaration or distribution outside Play) vs Gmail (needs verification and a paid security assessment). Decide from measured notification coverage.
- SMS-app rule: how to recognise business senders reliably, and whether to offer the SMS app in the allowlist by default (suggest: offered, not pre-selected, with the rule explained).